

Introduction to TouchDesigner

NVOID ART-TECH LIMITED

目录

序言	3
1 基础知识	7
1.1 信号流和连接线	7
1.2 创建 OP 元件.....	8
1.3 鼠标和键盘导航	11
1.5 使用外部文本编辑器.....	13
1.6 帮助	14
2 用户界面介绍	15
2.1 参数窗口.....	15
2.2 参数.....	17
2.3 运动控制.....	22
2.4 时间轴设置	23
2.5 分屏显示	24
2.6 资源查看器	27
2.7 搜索对话框	29
2.8 实时渲染	30
2.9 有用的快捷方式.....	31
3 TOP 元件	32
3.1 介绍	32
3.2 Movie 元件	33
3.4 Null TOP 和 Select TOP	35
3.5 编码器.....	36

序言

依靠无数的技术和技巧，一个工具软件，才在艺术家手中得以创造出优秀的作品。自从我听说 Elburz 打算写本关于 TouchDesigner 的书，就一直对本书充满兴趣。

Elburz 对 TouchDesigner 有着深厚的知识储备和深刻的理解，这是源于多方面的。比如论坛上对无数用户的帮助和交流；一些时间紧迫的重大项目的实施；用测试版的 TD 与我们 Derivative 公司一起工作；以及他纯粹的探索精神。

本书会是一本很好的办公桌或床边读物，一些之前我一直没有机会提出的一些 TD 背后的技术和概念，Elburz 在这里都提了出来。所以，本书既适合从头到尾细读，也可以跳着随便翻翻，像我就常能在里面发现宝贵的知识和深刻的见解。

非常感谢 Elburz 的这一工作丰富了 TD 社群。我相信它会产生巨大的反响。

Greg Hermanovic

Derivative 公司创始人

作者的话

写这本书有两个目的：

教一教 TD 088 的基础知识；给新手们创建一个群体协作型资源

第一个目的很直接。我们将看到各种 UI 元素，聊聊 OP 元件，探索逻辑工作流程，网络优化，接口性能，显示管理等等。我们将解释论证 TD 的核心元素，许多经常会遇到的问题将在这被提前处理。

经过本书的学习，我们将学习如何通过实例工程和视频教程，着手解决一些问题。这些课程将会很实用，我们将会从基础做起，汇总一些工程和有用的项目和模块。

第二个目的是对社群的要求稍高一点。我们真的相信这个社群，所以我们不是拿它来卖，而是希望每个人都能够免费使用本书（包括文本、视频和示例文件）。沿着这个想法，我们想更进一步，不仅让每个人能免费获取本书内容，而且每个人都可以自由的为文书添砖加瓦。

这意味着，任何人都可以随意添加、注释、更改、标记、删除或增加本书中的资源。全书已经用 LaTeX（译者：排版软件）写完，书中的代码，项目文件，例子，图片，视频，以及它所包含的一切，都将被托管在 GitHub 上（译者：托管代码网站），并将免费-在 Creative Commons（译者：一个知识共享组织）许可下-供任何人下载，分享和修改。

处于对资源质量的考虑，虽然每个人都能够对本书的知识库重新编辑，但我们会在整合进主资源库前审核所有的变更。所有贡献的人将被致谢，并加入到本书的“编辑人”部分。这样，一个新的用户只需要找一个 PDF，或下载链接，就能收到 TD 圈的最新知识。

我们真心希望我们 TD 社群能从事好这项工作，创建完善的资源，让初学者能从头学起。

写本书之前，清楚这本书的定位是很重要的。这本书不是 OP 参考手册。我们不会涵盖每一个参数或者用上每个 OP 元件。这本书不是用来替代 Derivative 维基上的资料，当作主要的参考资料来用的。我们将只使用并学习我们必须的东西。这本书并不能替代论坛上大量的有帮助的用户和元件。

最后，这些资源，是送给 TD 程序员和 Derivative 公司员工的礼物。是他们，无论在论坛上，还是在现实生活中，帮助我们走到今天。我们希望这个传统能与 TD 一起长存。

Elburz Sorkhabi & nVoid 艺术科技有限公司

TouchDesigner 是个啥？

许多使用者在开始时都曾花时间来回答这个问题。你会惊讶于用它来创建和执行简单的任务所花费的时间，也会惊讶于需要花费不少时间来实现一些在其他软件中已经现成的功能。

那么到底什么是 touchdesigner？答案很简单：touchdesigner 是一个视觉化的基于节点的编程语言。

让我们先从最重要的开始介绍，TD 是一种编程语言。TD 不是一个 APP 应用程序，像一般程序那样可以很容易的开始执行动作。TD 是一个很复杂深奥的环境，额，并且有不少坑。通过一段时间的实践，很多功能都能较快实现。因为本书的目标是创建可重复使用的模块，所以一个项目从一张白纸到完工的速度将会越来越快。然而 TD 仍然只是一个编程语言。仍需我们付出时间和努力来完成一个个任务。相当一部分可能还很有些让人头疼，所有这些都需要具备基本的解决问题的能力。

我要介绍的第二点是，touchdesigner 是基于节点的图形化编程软件。这意味着，我们不用打开一个文本文档然后一行行敲代码，TD 用图形化界面来通过节点创建程序。TD 的每一个节点或 OP 元件，执行一个具体的，小的，独立动作。实现一个整体功能，需要多个节点共同完成。它们的输入和输出连接在一起，来传递信息。有许多基于节点的编程语言，如 Cycling 74 的 Max/MSP，而让 TD 与众不同的是它的视觉特性。

TD 中的一切都有可视化的对应。所有 OP 元件都有查看器。一切的一切，无论是文字、音频、视频、控制数据等，以及执行的 OP 元件，都是可视化的。这点不同于任何传统的甚至是基于节点的编程语言，正是它让 TD 成为一个不可思议的世界。良好的视觉化相关流程的能力，实现每一步的方式，都将大大简化你学习如何实现整体任务。

获取本书

有多种方式来获取 TD 入门 088 版。由于已将本书转到 gitbooks，现在你可以从下面的链接下载 PDF，EPUB，或移动版的本书：

<https://www.gitbook.com/book/nvoid/introduction-to-touchdesigner/details>

然后记得从我们的网站下载软件包，其中包括一个 PDF 的书，所有的例子，.toe 文件和一些跟视频教程同步的项目文件。

链接是：

https://d31vryd1jmut49.cloudfront.net/Introduction_to_TouchDesigner.zip

高清视频教程在 Vimeo（译者:国外的优酷）的一个独立频道。所有的文件都可以从 Vimeo 下载来在线下观看。（译者: 然而你需要翻墙）

下面是链接：

<https://vimeo.com/channels/845218>

你也可以从 GitHub 下载源代码，用其中的任意部分编译。你可以通过对资源文件的重新分配来自由更改、编辑、添加、删除和修改文本。编辑完成后，提交你的更新，它们将通过我们的管理评审，并合并到主分支，你将因此被放到贡献者名单里，和你的贡献一起添加进本书。

有关此资源更多的一般信息，访问 <http://book.nvoid.com>

编辑本书

与以前的 LaTeX 版本不同，不必建立环境就能投稿本书。你可以使用普通的 Markdown（译者:标记性排版语言），你的更新会被资源库保存，gitbookS 会自动创建一个新版的网页、PDF、EPUB（译者:电子书）和手机版的书。

归属许可

本资源的授权依照国际 Creative Commons - attribution-noncommercial-sharealike-4.0 标准。

链接：<http://creativecommons.org/licenses/by-nc-sa/4.0/>

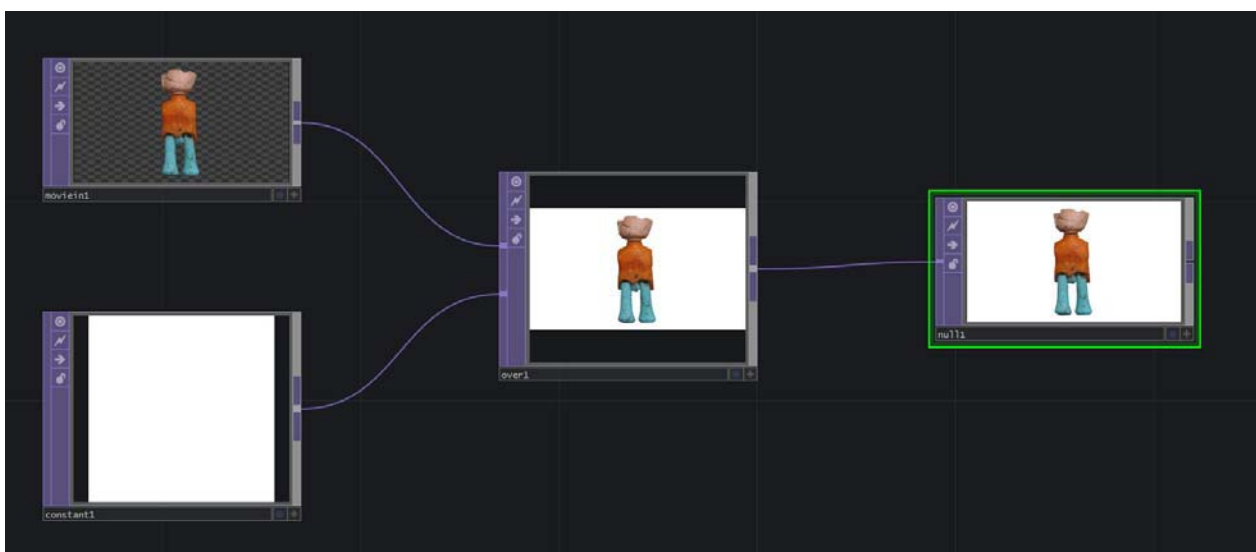
Thank you!

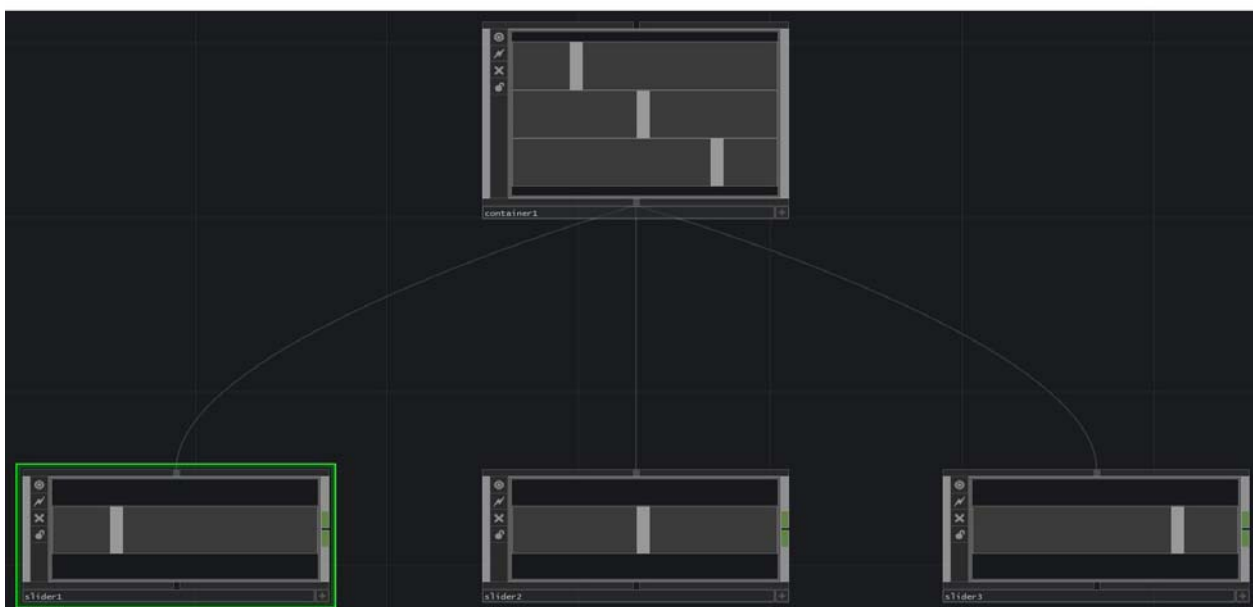
1 基础知识

1.1 信号流和连接线

连接 OP 元件是 TD 中最基本的操作。所有项目都是无数的 OP 元件连线组合而成的。每个 OP 元件自身完成一个特定的功能，当他们组合成网络，就能解决非常复杂的问题。

TD 中的数据都是从左往右传递。所有 OP 元件总是左边输入，右边输出。输入输出同时也遵循按顺序执行，从上往下。下图中，沿着左边开始的两个信号，从左到右，组件一个个被整合。有趣的是，这些从左往右拥有相同信号流的 OP 元件，依然遵从父子关系，从上往下传递。最上面是祖宗，最下面是孙子。再往下那张图的例子中，有一个由一些滑块组成的界面。在那张图中，三个滑块就是容器的子物体。





1.2 创建 OP 元件

创建 OP 元件有多种方法。每种方法都有它适合的使用场景。最简单的两种方法是点击“Tab”键，或者在背景界面上双击。

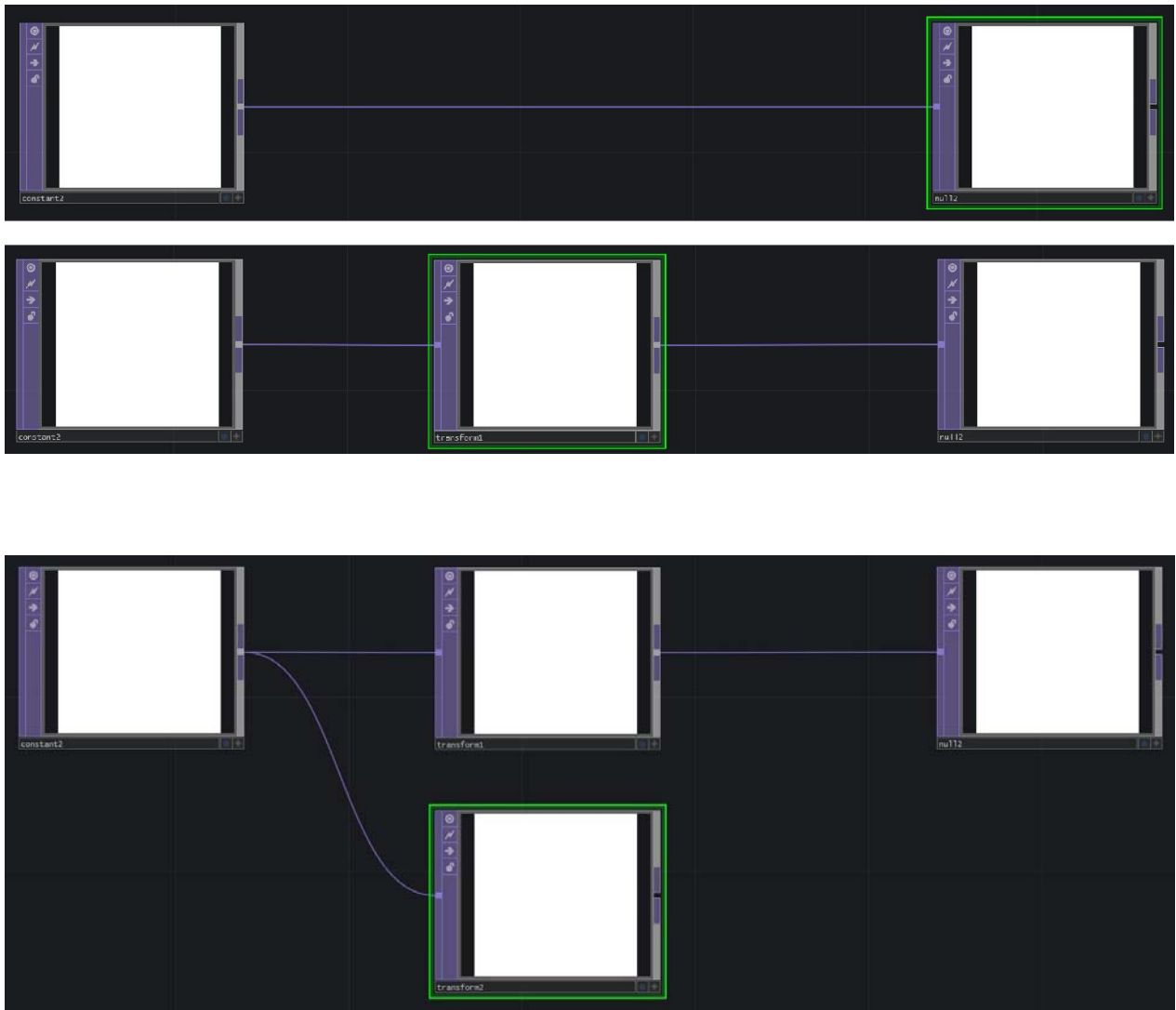
对已经有了连接线的 OP 元件，有两种方法添加相关联的 OP 元件。一种是右击现有 OP 元件的输出或者输出，这样就会直接创建关联性的新元件。当往一个已有的链路中添加元件时，这种方法尤其好使。

例如，有一个 Constant TOP 元件连接到一个 Null TOP 元件，这时，想在连个元件之间加入一个 Transform TOP。右键单击 Constant TOP 的输出端，或 Null TOP 的输入端，选择 Transform TOP，就在这俩中间插入 Transform TOP。

另一个办法是在现有 OP 元件的输出端点击鼠标中键，区别在于，右键创建的 OP 会自动插入已有的两个 OP 中间，而中键创建的 OP 会在原链路外，新建一个并联的分支。

通过右键连接线，点击“Insert Operator”或“Add Operator”，也能取得相似的效果。“OPInsert Operator”相当于右键 OP 的输出端口，并把它连入当下 OP 的链路，而“Add Operator”的作用就像是点击鼠标中键，创建一个平行于现有链路的分支。

在下图中的例子里，有一个 Constant TOP 和一个 Null TOP,再往下那张图里，这两 TOP 之间的连接线被右键单击后，通过插入 OP 的方式，创建了一个 Transform TOP。最后那张图里，是通过右键连线，用“Add Operator”（增加 OP）的方式创建的 Transform TOP。注意它是如何创建的平行于第一个 Transform TOP 的接线。



在创建 OP 时，有 2 个超有用的快捷键：“ctrl”和“shift”。打开创建 OP 对话框，按住“ctrl”，然后连续选择多个 OP，这些 OP 会按顺序从上到下添加进界面。这招可以把几个 OP 快速添加进项目。

按 SHIFT 也是一样的操作方法。区别是，这样创建的 OP 会从左往右直接串联好，这种方法可以快速创建已经连号线的若干 OP。

这两快捷键真心强大。当用来创建一个个连接好的 OP 时，尤其好使。举个例子，一个项目需要 3 条 OP 链。

第一条先是一个 Null TOP，后面接一个 Blur TOP，最后面接个 Circle TOP。

第二条先后是 Noise TOP, Edge TOP, Circle TOP。

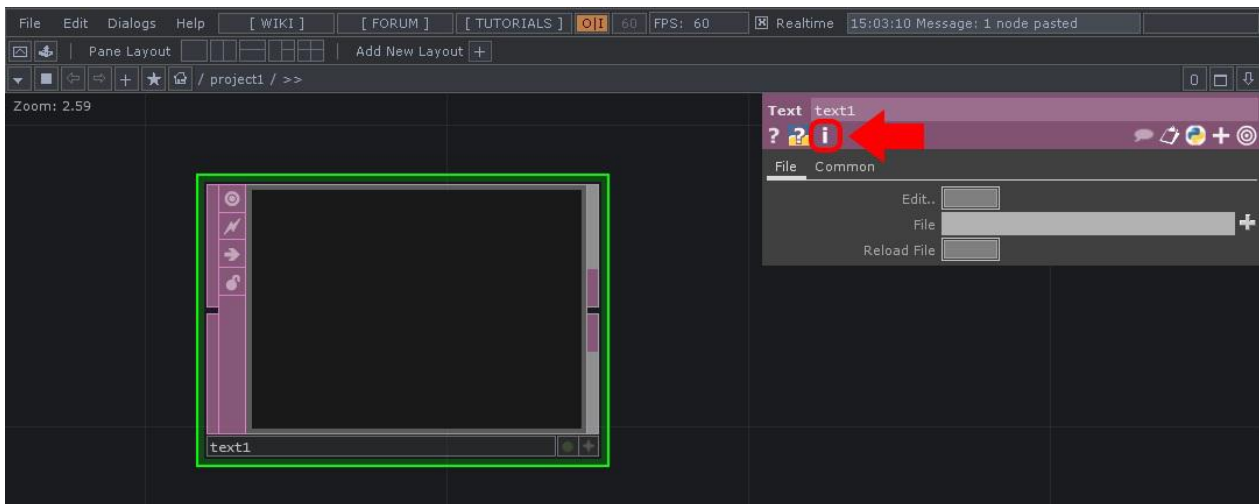
第三条是 Null TOP, Blur TOP, Movie In TOP。

咱们一步步过一遍这个步骤，来演示下这两快捷键的实际用法。

- 1.打开 OP 对话框
- 2.按住 Shift
- 3.单击 Circle TOP, Blur TOP, Null TOP , 创建第一个链路。
- 4.释放 “Shift” 键
- 5.按住 “ctrl” 。这将把下一个 OP 放在第一个 OP 下面。
- 6.保持按住 ctrl,点击 Circle TOP
- 7.释放 “ctrl” 键
- 8.按住 “shift” 键
- 9.点击 Edge TOP, Noise TOP, Null TOP,创建第二链路。
- 10.释放 Shift
- 11.按下 “ctrl”
- 12.同时 , 点击 Movie In TOP。
- 13.释放 “ctrl” 键
- 14.按下 shift
- 15.点击剩余的那俩 OP : Blur TOP, and Null TOP。
- 16.现在所有的 OP 都创建了 , 使用 “ESC” 键关闭 OP 创建对话框。

关闭 OP 创建对话框后 , 所有需要的 OP 已经被添加进项目 , 并连好接线。这些快捷键不光省得你创建每个 OP 元件都要打开关闭一次 OP 对话框 , 而且省的你给他们连线。

1.3 鼠标和键盘导航



点击图中的 i，可以获得更多所选 OP 元件的详细信息。

在 TD 工程中导航，有一些快捷键。其中两个是 i 和 u 键。

u 键会跳出当下组件，进入上一层网络。

i 键相反，进入一个网络或组件。

如果想看到某个网格中的所有 OP 元件，按 h。它相当于执行当下网格的 home 动作。

1.4 网络和路径

所有的 TD 工程都是由网络组成的。一个网络其实就是一堆 OP 元件。

网络里被封装了各种组件，比如 Container 组件，Base 组件，Geometry 组件等等。

网络这玩意能无限嵌套。

最上面那层，叫根层。

在根层里，会找到 TD 系统和 UI 元素。

你最好训练自己从项目一开始，就习惯于封装和组织网络。在网络编辑器的顶部，有个路径栏，里面会显示当下路径。



所有 OP 都有路径。这种路径和 Unix 文件的路径相似。

路径有两种，“绝对路径”和“相对路径”。

绝对路径是项目的根目录或者“/”。

相对路径是从一个 OP 指向另一个 OP 的路径。它开始于网络中引用的某个 OP,而不是根目录。

打开范例“Paths.toe”这个例子可以说明路径是咋回事。

TD 项目是从工程根目录下的那个叫“network1”的 Container 组件开始的。在 network1 里，有两个 OP。

“rel1”是个 Text DAT 元件。它的内容语句中，写里有两条路径。第一个是绝对路径。它始于根目录，指向元件。另一个是相对路径，它始于本元件，指向 network2 中的另一个 Text DAT，“rel2”。相对路径从当下位置指向目的地。从 rel1 到 rel2，路径只用穿过 network2，所以就是‘network2/rel2’。

注意 network2 显示的是它内部的某个元件，这一点将在后面的例子中详细讨论。现在我们要关心的是它的路径的用法。在 network2 的 Operator View 的 there 参数中，有一条路径‘./display’。display 是它内部某个元件的名字，‘./’则表示当前元件也就是 network2 的下一层。

在 network2 中，display 的内容被显示在它的上层容器中，另外两个元件则演示了路径的其他用法。

abs1 是绝对路径的一个例子。rel2 演示了它与 abs1 之间的相对路径，以及它与上层网络中元件 rel1 的相对路径。

rel1 是元件名称，../表示位于上一层网络。../可以叠加使用直到根网络，但还有别的更高效的方法。

1.5 使用外部文本编辑器

TD 中本身可以创建和编辑简单的 Python 脚本。不过随着脚本增长，用外部的编译器会更省事快捷。这样会有一大堆好处，我们简单列举：

- 代码行号；
- 分颜色的代码；
- 查找替换功能；
- 自动联想语句；

所以，在 TD 中广泛使用 Python,会让你的工作更高效。

在编辑菜单中打开参数对话框，进入 DATs 参数选项，在 “Text Editor” 设置中，点击文件浏览的图标，找到并选定编辑器的 exe 文件，打开，点击接受。

设置完成后，右键一个 DAT，然后点击 “Edit Contents” 选项，DAT 就会用刚才设置的编辑器打开。

有一个 “Table Editor” 参数，专门用来设置 Table 类型 DATs 的编译器。

这有两 TD 中用的比较多的跨平台编译器：

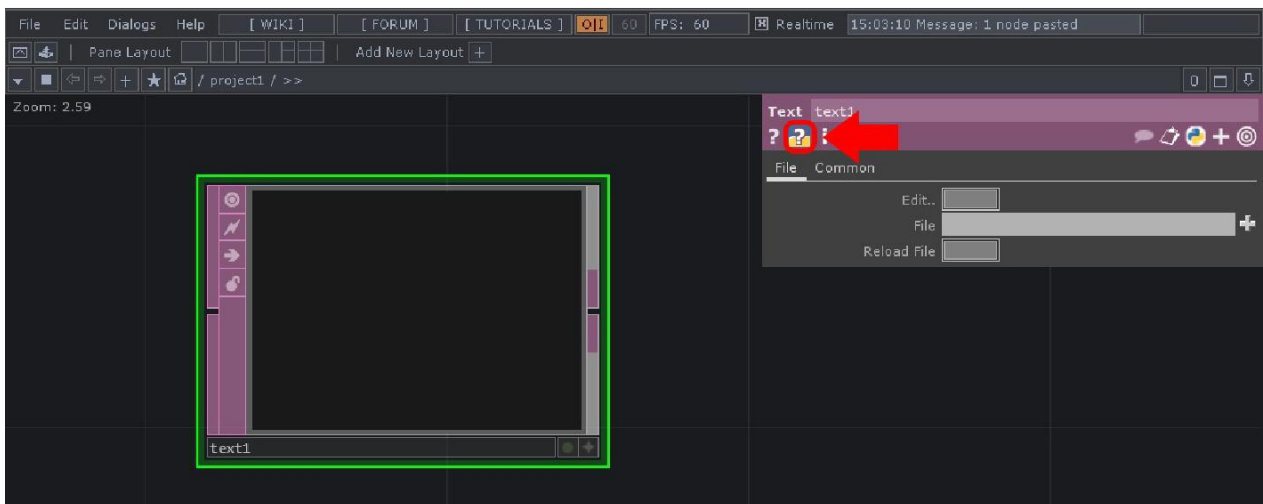
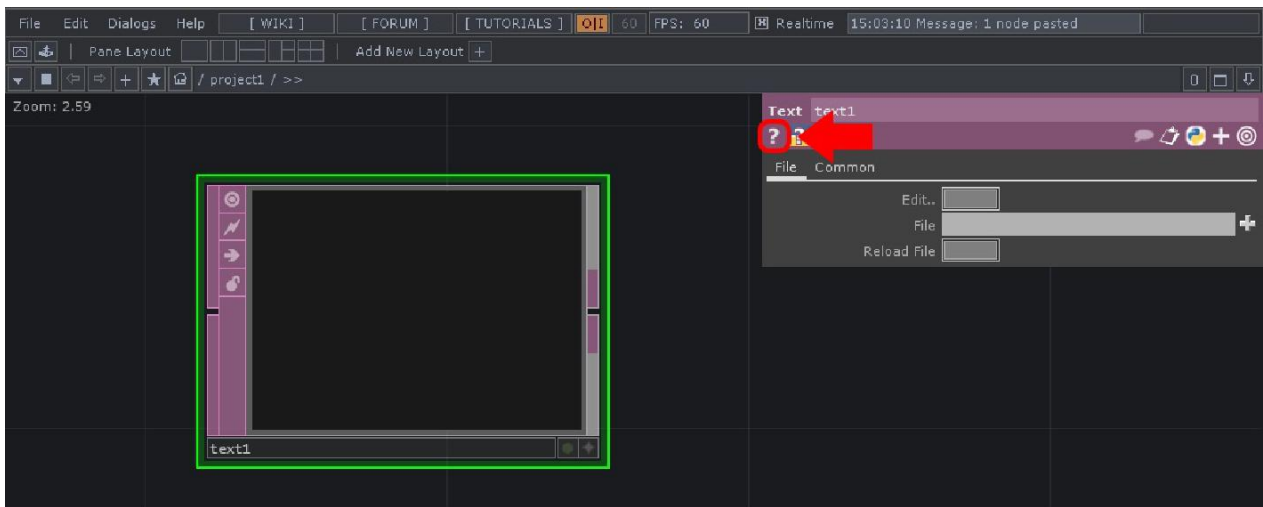
Sublime Text 3 <http://www.sublimetext.com/>

Notepad++ <http://notepad-plus-plus.org/>

1.6 帮助

关于程序和具体 OP 元件的任何问题，可以参考官方的 Wiki。每个 OP 有两个快捷方式，可以指向一个 WIKI 中的网页。

它们位于参数窗口，外形都是问号的样子。其中一个介绍本 OP 元件的用法。另一个带 Python Logo 的问号，是专门介绍跟这个 OP 相关的 Python 语句的。



2 用户界面介绍

2.1 参数窗口

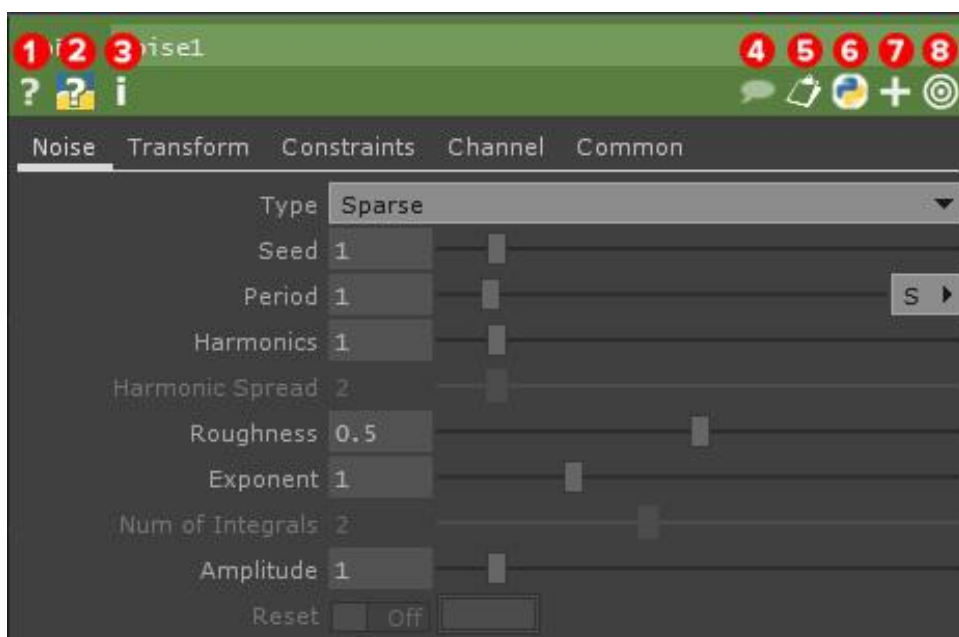
在参数窗口可以访问所有 OP 的参数。

有两种方法打开它。

一种是用“P”键，这样会打开一个停靠在窗体右上角的固定窗口。它会显示任何你当下选择的 OP 的参数。

另一种是直接右键 OP，选择 Parameter(参数)，这样会打开一个浮动窗口。这种方法的区别是，此时如果你选了别的 OP，这个窗口的参数还是之前那个 OP 的参数。

想要同时操作多个 OP 的参数时，第二种方法会很有效。虽然每个 OP 的参数设置都不相同，但它们都用相同的参数选项。下面是个某个 OP 选项的参数。



从左往右，这些参数分别是：

1. 元件帮助：在浏览器新窗口中打开 OP 的 WIKI 帮助页面。
2. 元件 Python 帮助：在浏览器新窗口中打开 OP 的 Python 帮助页面。
3. OP 信息对话框：显示 OP 程序的信息，效果等同于中键点击 OP
4. 备注：显示和编辑 OP 的备注
5. 复制参数：显示通过邮件点击菜单复制的参数
6. 语言：选择用 Python 还是 tscript 作为 OP 的脚本语言
7. 展开/合并参数：展开或合并显示这个 OP 的全部参数
8. 非默认参数：只显示被改动过，非默认状态的参数。

2.2 参数

有多种方式可以设置参数。根据情况，有些参数需要是静态值，另一些参数需要随别的变量或输入而变动。每个参数有三种类型。这三种类型完全不同，每种都决定了参数的运行方式。他们是：

常数类型

表达式类型

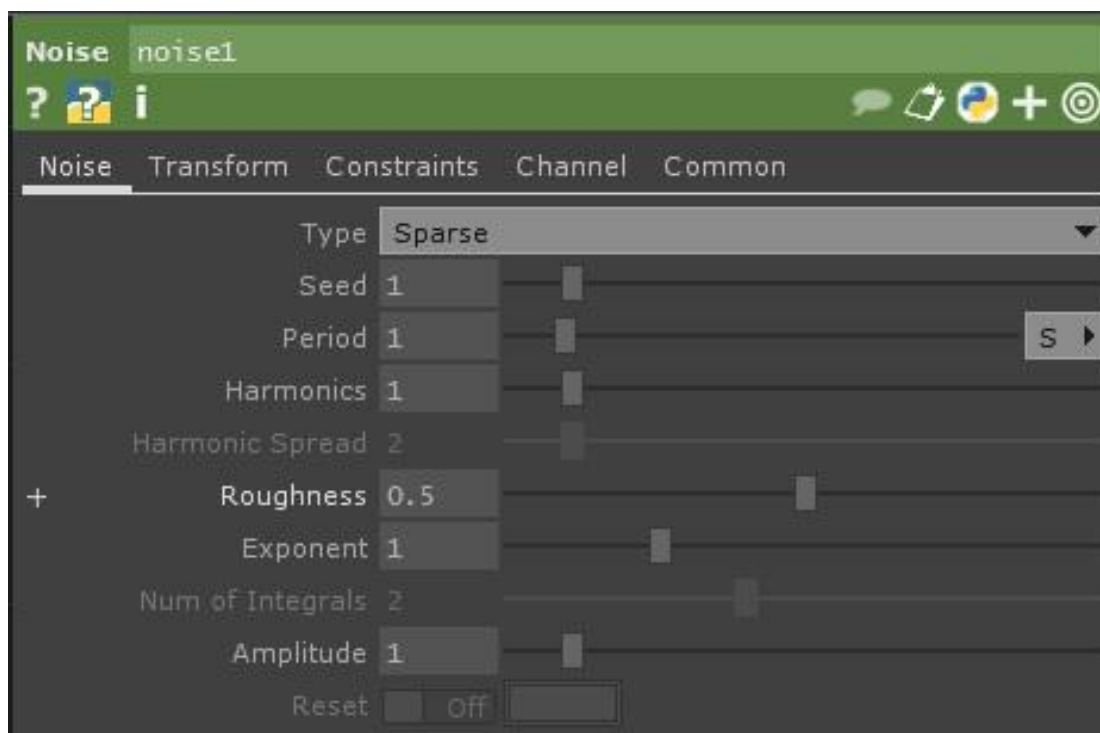
输出类型

大多参数的默认类型是常数型，它的数值区域显示为灰色。

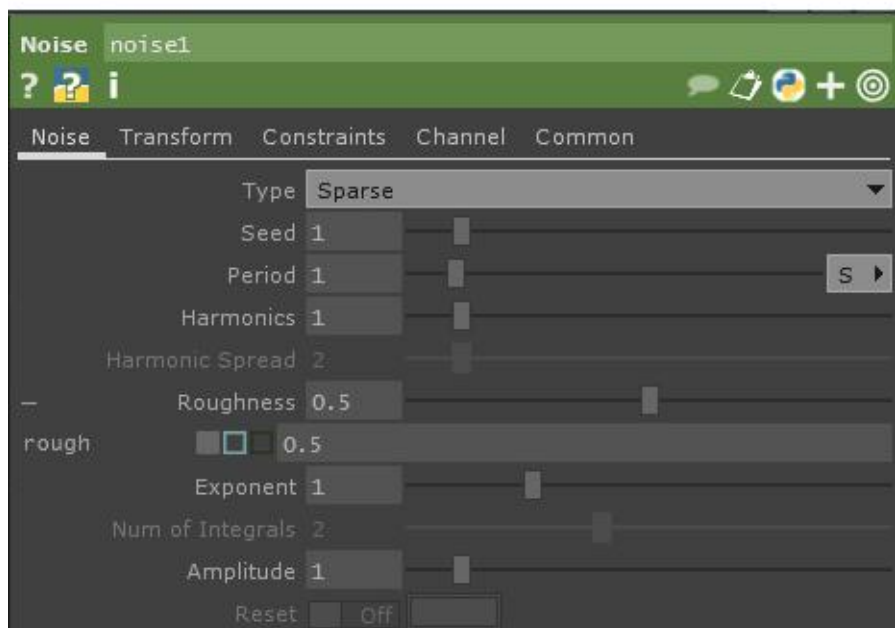
表达式类型用于 Python, tscript, 数学运算或脚本，颜色为深灰色和亮蓝色组合。

输出类型直接引用 channel 的取值，它的表现颜色为亮绿色。

每一个 OP 参数都可以在这三种类型之间切换，只要把鼠标悬停在参数名称上，参数名称附近会出现一个加号，就像下图：



点击加号，它会展开显示更多信息和选项，就像下面：



当这个参数被点击后，出现三个可变的元素。

第一个是这个参数脚本的名字。

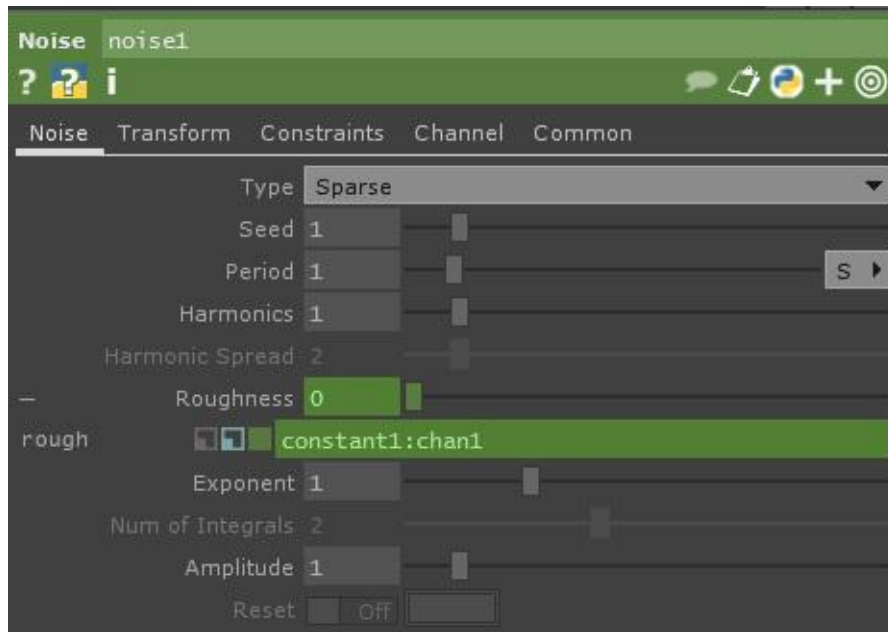
在 TD 的任何一种脚本语言中，任何时候引用参数，都需要脚本名称。这上面那个例子里，Noise CHOP 的 Roughness 参数的脚本名称是 'rough'。

在 Python 中设置 Roughness 的值为 1 的语句如下：

```
op('noise').par.rough = 1
```

第二个元素是三个不同颜色的方块。他们代表参数的不同类型，就像上面提到过的。默认为灰色的常数类型，点击蓝色方框，可以变为表达式类型。点击后，右边的参数值也会根据类型不同，变为对应颜色。

要是想变成输出类型，需要把某个 CHOP 元件的 channel 拖到参数上，然后输出类型的小方框会被自动选上，后面的背景色也变成了绿色。



第三个元素是数值区域。它显示所选类型的一些信息。

常数类型下，它显示当前值，并且可以被编辑和输入。

表达式类型下，它显示对应的 Python 脚本。这些表达式可以被输入和编辑。

输出类型下，

数值区域被一个冒号分为两部分。冒号前是来源元件的名字，冒号后是那个来源元件中的参数名称。因为这些值会被其他 OP 应用，所以在输出类型下，参数不能被编辑。

2.3 运动控制

TD 运动控制栏的功能和很多软件是一样的。咱们从左往右，把这些按钮过一遍。



1. 把时间轴拖到第一帧
2. 暂停时间轴
3. 反向播放
4. 正向播放
5. 后退一帧
6. 向前一帧
7. 在选定范围循环播放
8. 在选定范围播放一次，并停在最后一帧

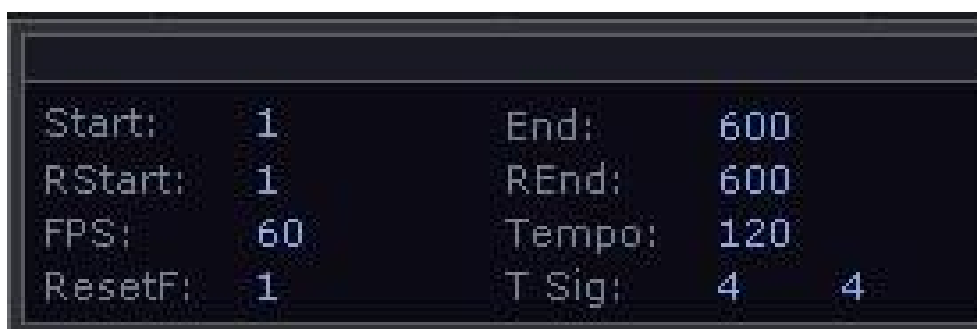
最常用的功能是开始和暂停。用空格键实现。

2.4 时间轴设置

除非有视频或动画被锁定到时间轴上了，否则一般来说，不用经常操作时间轴的设置。时间轴设置在屏幕左下角，重点是要知道，这里可以改变工程运行的帧速度或节奏。帧速度决定了项目渲染每一帧的速度，默认是 60 帧/秒。也就是说工程会尽量保证每秒渲染 60 帧画面。

节奏决定了项目每分钟的节拍数，Beat CHOP 这东西会用到它。

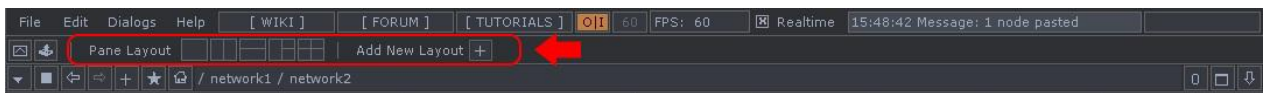
时间轴设置常用在媒体或者动画需要被锁定在一个一致的时间轴上的情况。帧控制包括 'Start' 和 'End' ,用来控制时间轴的第一帧和最后一帧。同样，'RStart' and 'REnd'则是用来控制时间轴循环播放的开始点和结束点。通过这些设置，既能创建一个横跨整个时间轴的 4000 帧动画，也能实现循环播放时间轴的一小部分。

A screenshot of the Time Axis settings panel in TouchDesigner. The panel has a dark background with light blue text. It contains several parameters arranged in two columns. The first column includes 'Start:', 'RStart:', 'FPS:', and 'ResetF:'. The second column includes 'End:', 'REnd:', 'Tempo:', and 'T Sig:'. The values for these parameters are: Start: 1, RStart: 1, FPS: 60, ResetF: 1, End: 600, REnd: 600, Tempo: 120, and T Sig: 4 4.

Start:	1	End:	600
RStart:	1	REnd:	600
FPS:	60	Tempo:	120
ResetF:	1	T Sig:	4 4

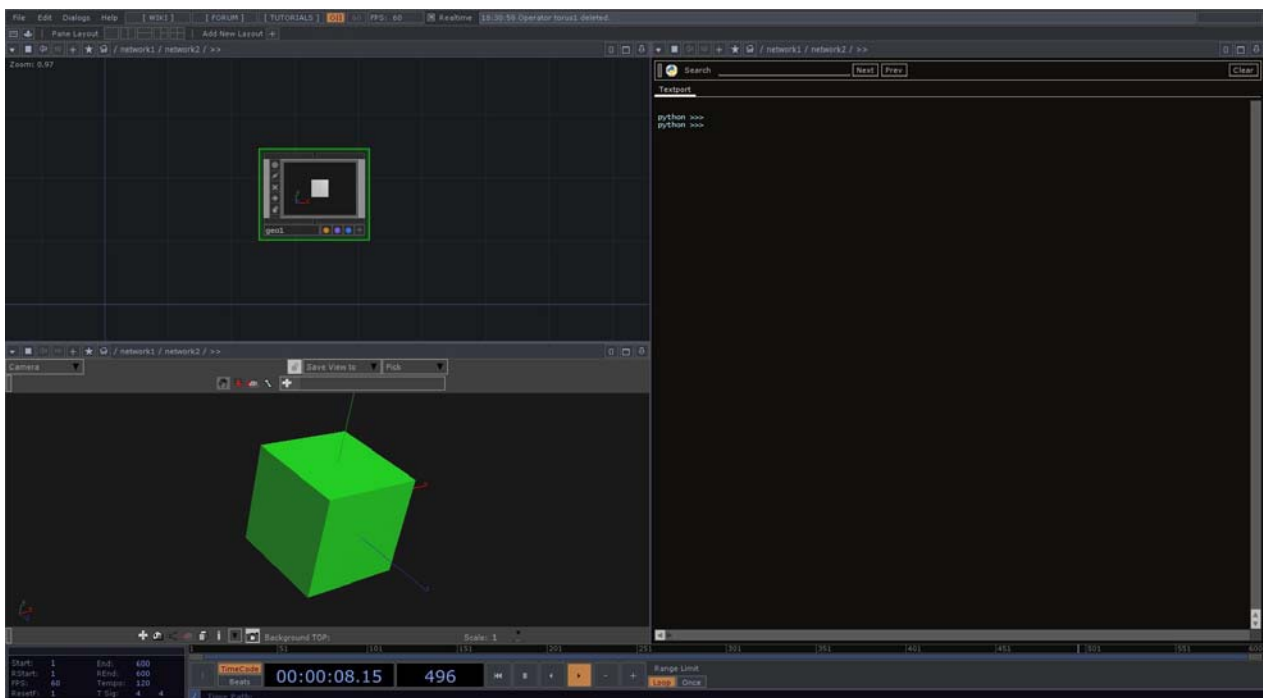
2.5 分屏显示

在网络之间来回移动和跳转的时候，经常用分屏可以节约很多时间。你要是穿过三层网络来改变一个参数，哪怕只是跳回去看一下变更，都会很浪费时间。而用分屏功能可以把窗口分割成任意块，这样就不用跳来跳去了，每个面本都能保存以供将来使用。



上图表示了默认可用的几种预置的分屏类型。预置提供了几种快捷的标准配置，包括左右分，上下分，分成三块以及四块。保存设置也很简单，点击'Add New Layout +'按钮，输入名字，点 OK。保存布局不止保存分屏的位置和尺寸，也会保存每个分屏的类型。

分屏面板可以用来显示独一无二的内容，不论是对话框，网络，还是视角。可以灵活的混合和比较组合的视角或网络编辑器。下图中，左上角的面板是个网络编辑器，右手边是个 Textport, 左下角是个几何视角。同样，保存布局不止保存了面板的排列，也保存了面板类型。当搭建一个有很多元素的项目时，这一点很有用。当需要上面这些设置之间跳转或是进入一个小的网络编辑器中时，会节约很多时间。



关于面板分屏的快捷键有这些：

1. Alt + [: 从鼠标处垂直切分面板
2. Alt +] : 从鼠标处水平切分面板
3. Alt + Z : 关闭鼠标所在面板

2.6 资源查看器（组件库）

资源查看器可以被看作是一个组件库。资源查看器用来汇总'.tox'文件（或者 TD 组件文件）。这些文件包含有一个 OP 组件，而这种 OP 组件保存着一个网格或者 OP 元件。这意味着，一些常用的 OP，网格，组件，脚本，可以被创建进一个单独的 OP 组件，存为'.tox'，并可随时调用。

打开资源查看器，然后浏览这些已经存在的可用的.tox 文件。空项目的资源查看器默认是打开的，停靠在窗体左侧。如果要使它成为一个悬浮窗体，用'Alt + L'。让我们试试其中一个已经预先构建的组件。

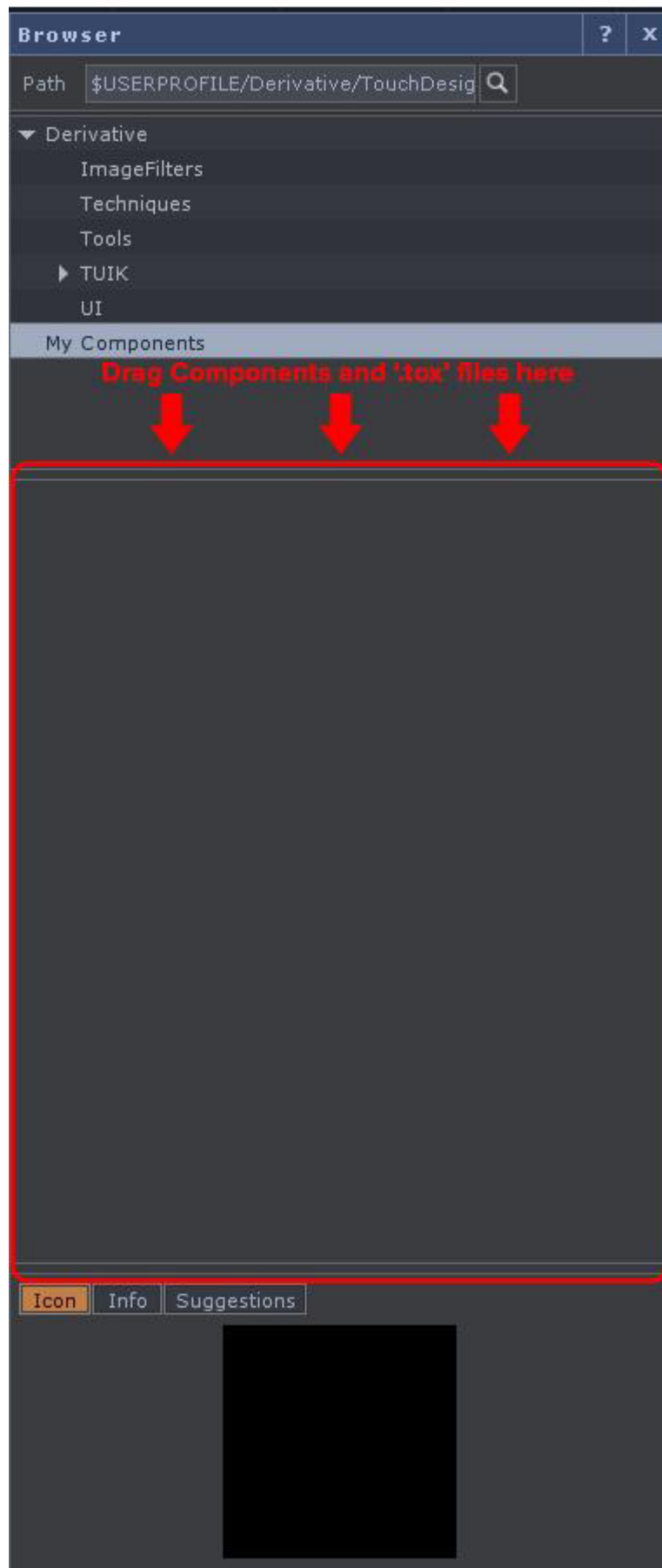
在'Derivative'区域，找到'Tools'，然后把'BlendModes'组件拖放到一个新项目中。观察'BlendModes'组件的 UI，很显然，它里面有很多东西在运行。在深入了解前，先连接两个输入，试着操作下'BlendModes'组件。你可以通过导入两个不同的图层来观察不同的叠加效果。这个工具很实用，而全部操作只是在资源查看器中的一个拖放动作。

本书的目标之一就是创建一些工具，使它们可以添加到资源查看器，以便可以被经常使用。有两种方法可以将组件添加到资源查看器中。

第一种是拖放。在查看器顶部选择'My Components'，然后拖动网格中的任意组件并放进资源查看器的下面，它就会被添加进'My Components'库。

第二种方式是从 Windows Explorer 中拖动一个已经保存的'.tox'文件，把它放到前面提到的区域。下图准确描述了组件应该被放到哪里。

Introduction To TouchDesigner



2.7 搜索对话框

当项目变得越来越复杂，充斥着无数的 Python 脚本或是嵌套的网络，搜索对话框会变得超乎想象的有用。它可以用于很多区域，查找很多东西，而且根据需要，可以选择模糊搜索或者精确搜索。按 F3 或在顶部的'Edit'菜单中打开搜索。

常规搜索不仅能查找 OP 元件你还能查找 Python 代码。通常，Python 用来改变 OP 参数值。有时候，在一大堆 Python 脚本里，常常不易找到一些特定行的代码。

比如利用下面这行代码可以找到所有与名为 transform 的元件的参数相关的代码信息。

```
op('transform').par
```

从结果中筛选代码比从网络中笼统寻找要容易的多。

“高级”搜索可以搜索任何名称、OP 类型、注释、标志等的组合。

当处理老项目时，常常要花不少时间来回忆当时构建的那个，复杂逻辑网络的内在工作流程。这时就用模糊的名称和类型搜索，会节约很多时间。比如，在一个高度嵌套化的系统里，在深层中的某个地方，可能有个影片，名字中有'movie'几个字母，是 TOP 类的元件，这些 OP 类型信息，和名字的片段，可以用来进行一次精准的搜索。

当获得搜索结果，每个结果都可以在点击后，在一个新的浮动窗口中打开。这些结果可以在这里被快速预览，简单编辑，并不会影像当前的工作区域。

2.8 实时渲染



实时渲染能合理地分配资源，在 GPU 不足时也可以进行编辑。当它被激活（默认激活），TD 会优先使用真实时间。举个简单例子，如果一个影片长 30 秒，不论发生什么，TD 会让他播放 30s。如果因此导致帧速度被降低，TD 会试着用虚拟时间。这种模式在实时装置和高性能要求的工作中经常用到。

当实时渲染被关掉，TD 将优先渲染真实时间。在上面提到的例子中，如果关掉实时渲染，TD 会按它需要来运算渲染每一帧，脱离真实世界的时间来显示每一帧。这个模式在导入复杂的动画或 3d 模型时很有用。想想下，这个模式跟 Adobe After Effects 渲染时的实时画面比较像。

2.9 有用的快捷方式

当鼠标悬停在网络上方，下面的快捷键可以使用：

'P' 打开/并关闭选定 OP 的参数窗口

'O'在面板左下角打开和关闭当前网络的概览。

'C'打开和关闭资源查看器。这可以为选定的 OP 添加一个彩色的轮廓，以方便识别

'A'允许与 OP 在状态窗口和编辑窗口之间进行切换

'B'忽略或取消忽略选择的 OP

'H'执行总览网络的动作，实现在屏幕中看到当前网络的所有 OP

'Shift + H' 在屏幕中查看所有选择的 OP

'R' 切换 OP 的渲染标志（如果有）

'D' 切换 OP 的显示标志（如果有）

'Control + C' 复制选择的 OP

'Control + V' 粘贴复制的 OP

'Control + Shift + V' 在鼠标处粘贴复制的 OP

3 TOP 元件

3.1 介绍

图片纹理元件，即 TOP，几乎会在每个项目中用到。他们用于处理影片播放、3d 渲染、合成、硬件视频输入输出等这些动作时的图形操作；被用于表现任何输出到监视器、投影机、或 LED 上的东西。

3.2 Movie 元件

在 TOP 元件里，Movie 是最常用的种类之一。它的功能是把资源加载进 TD。它能加载多种不同资源，从静态图片到各种视频编码格式。下面是一部分 Movie 常用的文件格式。

.mov、.mp4、.avi、.jpg、.tiff、.png

还有很多其他支持的格式，在 wiki 的'File Types'页下有它们的清单。

在 Movie 中，有一些很棒的功能，可以大大减少那些令人头疼的不同帧速度的资源的采集和输出。一个主要的功能是，Movie 的目标是按真实时长播放资源。举个例子，如果项目设置是 60FPS，有一个 10S 的 30FPS 的资源，它会播够 10S 的时长，而不管项目和资源之间的帧速度的差异。相反也是一样。一个 60FPS 的 10S 资源，在一个 30FPS 的时间轴上播放，也一样会播放 10S。在上面这两种情况下，帧数被增加或减少，来匹配真实世界的时长。因此在某些情况下，使用插入帧会成为一个好办法。

3.3 预加载影片

当创建实时程序时，帧速度的下降会严重影响体验感受。后面的章节我们会讲到性能优化，除此之外，还有很多预防措施。

预先加载和卸载 Movie 就是一个办法。这个简单的步骤常常被新手忽略，因为这个最简单的办法需要脚本。

打开'Preloading.toe'的例子，这个例子里有三个按钮。

在'moviein1'元件的'Tune'参数下，有一个'Pre-Read frames'参数。

'Preload'按钮用下面的 Python 函数来预加载'Pre-Read frames'值所代表的帧数。

```
op('moviein1').preload()
```

'Play'按钮开始播放 Movie。'Unload'停止播放'moviein1'，卸载影片，释放系统资源。下面的 Python 脚本可以完成这些工作。

```
op('play').click(0)
```

```
op('moviein1').unload()
```

播放影片前，最好预加载，否则就等着播放的时候卡成狗吧。

3.4 Null TOP 和 Select TOP

相比那些费资源的 TOP,比如 Blur TOP，另外一些 TOP 是不需要占用系统资源的，能随便用。比如 Null TOPs 和 Select TOPs。尽管这两个 OP 不改变任何元素，但在创建高效的流程上非常有用。

一个布局合理的网络常被放在 Null TOP 和 Select TOP 里，而不是任由连线交错重叠，难以辨识。

打开例子'Null_1.toe'和 'Null_2.toe'。第一个文件是一大堆 TOP 混在一起。这个项目没有考虑网络布局，接线被 OP 和其他接线重叠覆盖，难以判断 OP 之间的具体关联。

在第二个例子里，先用一些 Null TOP 汇集所有信号，然后再将这些 TOP 组合起来。这些 Null TOP 可以作为一个节点，在快速浏览时，能更容易的追踪 OP 之间的联系

Select TOP 也是一样。当使用嵌套网络，使用 Out TOP，在容器之间连线，会产生和上面一样的情况，网络很快变得难以辨认。Select TOP 则可以快捷整齐地引用其他 TOP。打开例子 1，这个例子演示了用 In TOP 和 Out TOP 会额外导致怎样的杂乱。这个例子也只是复制了 12 遍影片。回头咱要是需要复制 100 份咋办？这就是用 Select TOP 的方便之处。

打开例子 2，这个例子成倍增加了所复制的组件的数量，同时显得更清晰明了。更有趣的是用 Select TOP 创建的动态选择系统。这比之前的手动方法要高效的多，允许用 Select TOP 的 Select 参数中的 Python 脚本，根据名称，来自动引用从上面网络中复制来的相应 TOP。在这个观念上更进一步，在 Replicator 复制器的使用上，如果 master 中能有一个 selectDAT，每个新产生的子节点都会引用外部数据。如果觉得这里例子很难，不要担心，后面的例子会涉及复制和脚本。现在，最重要的是知道，通过使用 Select TOP 和简单的脚本，组件会变得相对可持续，而且便于维护。当我们需要复制更多项目，会变得跟向表格中添加行一样容易。

3.5 编码器

视频播放是一个繁杂的过程。明智的做法是花时间尝试不同的编解码器，看哪种在视觉效果和性能的均衡上最适合该项目。

在开始操作具体的编码器前，了解编码器和容器之间的区别很重要。编码器是音视频文件合格的总称，它容易让初学者弄混。因为容器可以容纳多种编码器。

编码器是用来压缩和解压的。它有两个主要任务，第一个是压缩视频数据以便于储存和传输；第二个是解压视频数据来播放。因为这两种不同任务的存在，每个编码器有不同的侧重。一些倾向于把文件压缩成小体积，便与传输；而另一些侧重压缩为画质高，适合长期保存的影响。不同项目有不同需求，有时候，目标是用最高质量播放一个内容，另一些时候，必须降低画质来同时播放多个文件。为项目选择正确的编码器需要经过一些测试和思考，但可能会花不少时间。

容器的作用，跟它的名字一致。它可以容纳压缩的视频，音频，以及所有一个影片需要解压个播放的数据内容。在 TD 中有多种不同的容器，不过相对于编码器，他们对整个项目的流程影响区别不大。

当不同种类的容器和编码器组合使用，事情会变得复杂。想像下，有个叫 'test_movie.mov' 的视频文件。在一个项目中，这个文件是个 QuickTime 容器中的一段 Animation 格式压缩编码的.mov 文件。有趣和让初学者迷茫的是，在另一个项目里，它是一段 H.264 压缩编码的文件。更混乱的是，这个 H.264 文件可能还在一个 MPEG-4 容器中，用 '.mp4' 做后缀。

抛开这些混乱，目前的 HAP 家族中用很多流行的编码器可选。H.264, Animation 编码, 还有 Cineform。每个编解码器都有自己的优点和缺点。下面是这些编码器的优缺点对比。

HAP 家族：

Pros

优点：

可以播放极高分辨率和高帧速率视频

极低的 CPU 消耗

HAP Q 是视觉无损压缩

极低的 GPU 消耗

缺点

大文件大小

难在窗口上编码文件

必须使用固态硬盘或 SSD RAID0 文件播放

主要瓶颈是硬盘读取速度

H.264

优点：

可制作轻量级视频

视频画质相对压缩程度，做的最好。

磁盘使用率低

缺点

需要大量的处理器内核来播放极高的分辨率或高帧速率。

如果在编码中不正确的话，会感到颜色分层化

比特率随内容变化明显

长宽分辨率都是 409

难以创建 alpha 通道

Animation Codec

优点

100%无损文件

优先考虑质量

自带支持 Alpha 通道

缺点：

文件很大

对硬盘和 CPU 的要求都很高

比特率随内容变化明显

Cineform

优点

恒定比特率

高图像质量

自带支持 Alpha 通道

缺点：

文件大

必须购买 cineform 软件编码

4 CHOP 元件

4.1 介绍

数据通道类元件，Channel Operators，缩写为 CHOP。是一个负责各种通道类数据处理的元件家族。它可以处理诸如手势交互、音频输入、动画关键帧、硬件输入（Kinect 体感相机、LeapMotion 手势识别设备、Oculus 虚拟现实设备、Pen Table、键盘、麦克、等等）、DMX 调光、MIDI 电子音乐以及 OSC。

这些元件处理输入、运算、输出，并将数据与各种视听设备连接。

例如：

Mixers 调音器、MIDI 控制器、合成器、DMX 调光控制器、Kinect、运行有 TouchDesigner 的其他电脑、扬声器、以及其音视频应用如 Max/MSP，Ableton Live，Resolume。

4.2 通讯方式

MIDI 与现有的很多软硬件有不错的兼用性。数字音乐工作站-DAW,例如 Ableton Live, Avid Pro Tools, Propellerhead Reason 等等，都支持 MIDI 的输入输出。它是一种快速稳定并经过实践考验的协议。调音器通常配备 MIDI USB,它的输入包括按钮、faders、琴键、触摸板、滚轮、节拍器、电位计。

程序运算系统诸如 Cycling 74 Max/MSP, PureData, Native Instruments Reaktor 等等，都已经支持 OSC 通讯。得益于现代网络通讯技术，OSC 有着比 MIDI 更高的性能和更好的架构。OSC 消息可以在 UDP 和 TCP 连接中传输，因此它很容易实现实时长距离的网络传输。目前，OSC 是软件和计算机系统之间通讯最常用的方式。

DMX 是供灯光和控制器之间使用的协议。许多 DMX 设备都有多路调光、灯光跟随、RGB 通道、自动化电机等等。许多灯光控制器用 DMX 与其他设备或电脑通讯。当你打算用这些控制器和平台组建项目时，记得认真参考他们的手册。一般来说，就算有些功能这个项目用不上，你也应当有所了解。有多种方式优化 DMX 的数据收发流程，这些将在后面提到。

同步输入类 CHOP 和同步输出类 CHOP,通常用于 TD 的内外接口的帧同步。他们用 OSC 做底层的通信协议。这两类 OP 通过在每一帧通讯同步状态来工作。当所有同步设备确认，自己已经完成本帧的渲染，他们会同时进入下一帧。通过在每一帧重复这些事件来保证设备间的画面同步。

4.3 音频输入与输出

音频有多种来源和多种处理方式。TouchDesigner 能够使用音频文件，视频文件，外部音频接口，网络音频流等资源，来获取并处理音频。甚至也可以无需来源，自己生成音频。

大多涉及声音设计、音频音轨的项目都会有专用的音频文件。TD 能够读取和播放很多标准的音频文件格式，如 MP3,WAV,AIFF。通过 CHOP 元件中的 Audio File In 和 Audio File Out,这些音频文件可以被循环、修建、重建、插入，以供各种设备使用。

Audio Movie 型 CHOP 可以从视频文件中播放音频文件。不同于从文件中播放音频，该元件引用了一个 Movie In 型 TOP 元件。这样可以使音频与视频保持同步，并有一些参数来使音频更好的匹配视频。

TD 兼容多种外部音频接口，详细的兼容列表，最好去参考 TD 的维基百科和论坛。

这些外部设备可以提供各种模拟和数字音频的输入和输出。人声、乐器、摄像机、调音台、计算机，都可以作为音频的输入源。TD 中与外部音频接口通讯的是 CHOP 元件中的 Audio Device In 以及 Audio Device Out. 他们分别处理输入和输出。另外还有个 Audio SDI 元件（SDI 指的是 Serial digital interface，此元件只有在购买 TouchDesigner Pro 版本之后才会出现），专门跟英伟达的 Quadro SDI 通讯，采集外部音频。

TD 能访问两种音频驱动。DirectSound 是基于 DirectX 的成熟驱动，被大量使用。而在 TD 088 版后，加入了 ASIO，它可以改善 DirectX 的一个主要缺陷-必须基于 windows。它能绕过系统，直接与音频设备通讯，从而获得更低的延时。

当在 TD 中同时架起音频输入元件和音频输出元件这两门大炮，传输音频对你来说就像传输数据一样简单了。

4.4 采样率-Sample Rates

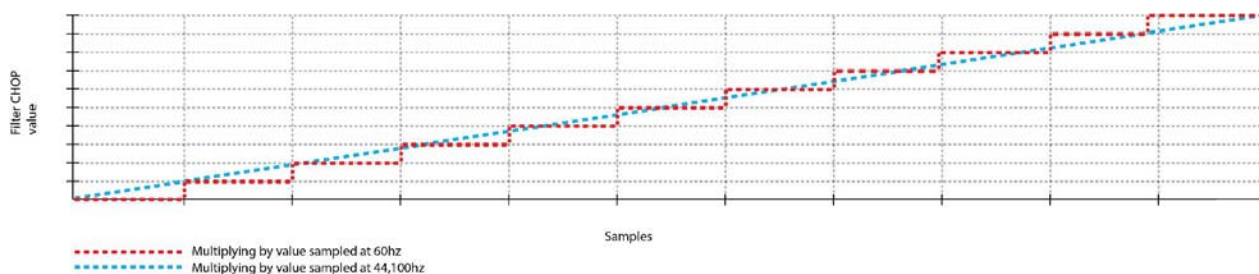
大部分程序不会体现出场景背后的函数与运算，所以大多数并不习惯用数字的方式处理音频。然而音频的本质，就是被高速运算的数据。认识到这一点，是能够用 TD 处理音频的基础。

打开'Sample_rates_1.toe'，这个例子是音频应用中的一个基本功能:静音。它用 button 的输出值(1 或 0)，和音频流数据相乘来控制静音。

咱们再增加点难度，让音频开关加上淡入淡出效果。打开'Sample_rates_2.toe'。

这个例子在上一个的基础上，增加了两个元件。一个是 Filter CHOP，它可以在按钮的两种状态之间创建一个平滑处理后的数值。另一个元件是 Resample CHOP。

初学者常常会忽略不同原件之间的采样率的差别。但想要创建可自由编辑音频，必须了解这一点。比如 Oscillator CHOP 的采样率是 44100 次每秒，Filter CHOP 采样率是 60 次每秒。这就意味着，当这两者联用的时候，不可能是 1 : 1 的采样比，在进行乘法运算的时候，并不会变成一个平稳上升的直线。更准确的说，前者的数据量是后者的 735 倍。也就是说，Oscillator 每过 735 个音频单元，Filter 才会有次声音上的变化。就像下图中，蓝线是 1 : 1 的采样比，红线是 735 : 1 的采样比。



再看上面这张图，当这两个不同采样率的通道相乘，有一个很明显的阶梯变化。很多 CHOP 元件用 FPS 帧速度作为他们的采样单位。到项目的帧速度被设置成 30FPS 的时候，阶梯将会变得更夸张，比率将变成 1470 : 1。也就是说在一个帧速度为 30 的项目里，每过 1470 个取样，才有一个会产生声音的变化。

上面的例子强调在项目中需要注意元件的采样率，以及按需求使用 Resample CHOP。除此之外，也有一些情况需要让输入输出数据使用不同的采样率。

4.5 时间片段化处理机制-Time Slicing

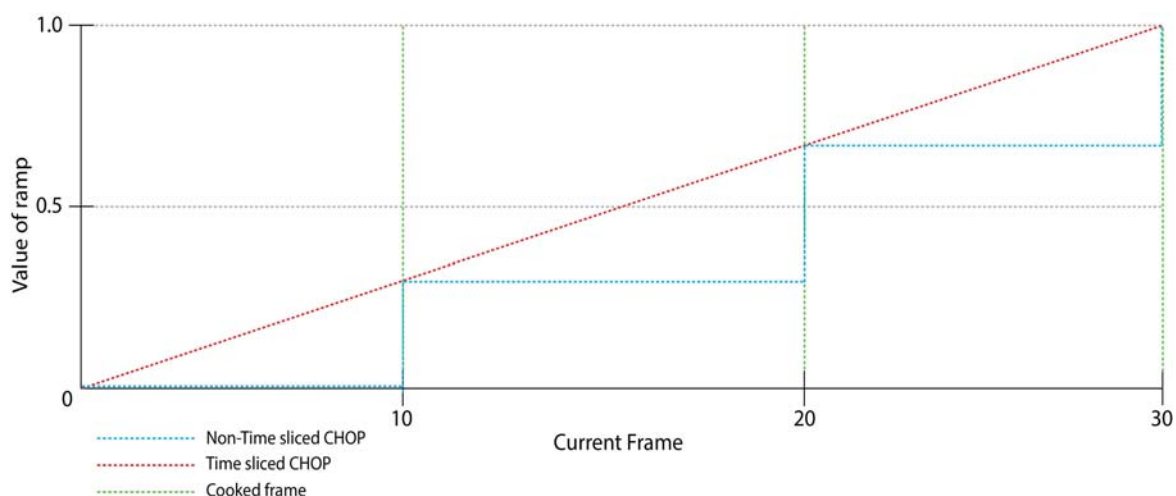
时间片段化是 TD 里面一个很独特的处理机制，我必须说，这很难一下理解明白。

一个片段单位是最后渲染帧和当前渲染帧之间的时段。把时间片段想象为一个动态的值，如果一个项目稳定运行在 60 帧，那么时间片段就是 1（因为并没有掉帧）。如果这个项目的实时渲染跟不上，每次渲染中间都丢 10 帧，那时间片段就是 10（当前渲染帧和上一个渲染帧之前实际上差了 10 帧）。

当存在掉帧现象时，时间片段化处理机制的存在就是为了平滑输出的 CHOP 数据。简单地说，时间片段化处理机制会在 CHOP 渲染的时候去考虑每个事件片段的长度。这种机制可以看作一种自适应处理，当时间片的长度增加，CHOP 会根据丢失帧的数量做出补偿，调节帧数量，来实现平滑的输出。形成鲜明对比的是没有做时间片化处理的 CHOP。他们在最后一帧处理完数据后，不管中间丢了多少帧，都会直接跳到下一帧的数据。只有 CHOP 元件能使用时间片段化处理机制。

在上面的例子里，如果时间轴以 30FPS 的速度持续运行，每个时间片段就是 1 帧的长度。如果在一秒钟内或者说 30 帧内，有两条线路的数值都是从 0 到 1，输出都是平滑型线路。但由于某些原因，每 10 帧才有一帧能被处理，两条线路的结果就会大不相同。在没有时间片化的线路中，在处理的那一帧，由于中间帧的丢失，数据会明显跳跃。有做时间片化的 CHOP 会注意到每过 10 帧才有一帧能被处理，所以它会在当前帧和最后一帧之间插入中间帧。这样就可以保证不论发生什么，数据都是平滑的。

下图可以说明上面的例子。红线是时间片化后的输出，蓝线是没做片段化机制输出的帧，绿色垂直的线是渲染过的帧。



5 DAT 元件

5.1 介绍

DAT 元件主要用来处理数据。他们可以编辑、分析、创建、收发多种类型的数据。比如文本字符串、列表、Python 脚本、XML、JSON、MIDI、串口、OSC。

项目中的逻辑处理，主要就靠 DAT 元件和 Python 脚本。比如分析列表数据信息，管理其他元件状态、通过外部通讯执行复杂任务，等等。后面会有一些这方面的例子。

TD 可以看作是一个模块化的 Python 编程环境。复杂、长篇的 Python 工程和函数，在这里被拆分，变成轻量化、模块化的片段。就像 TextDAT 元件那样。这种设定易于学习、维护、和扩展的特性，大大方便了分享和项目协作。

5.2 通讯方式

TD 原生支持 MIDI、OSC、TCP、UDP、UDT 以及 Websocket，所以它能跟多种系统如 Web 应用和服务、其他计算机系统、显示控制硬件等通讯。

CHOP 元件中的 MIDI、OSC、DMX 协议在 DAT 元件中也适用。

TCP 是因特网的标准通信协议。它是一个面向连接的协议，在通讯双方中有明确的客户端和服务端，并且在传输数据前，先要进行握手连接。这种连接是可靠的，通讯双方可以校验确认所有数据都已经被接收，也就是没有数据丢失。TCP 是一种顺序流通讯，发送方按一定顺序发送，接收方也按一定顺序接收。

而 UDP 是一种无连接的协议。数据发送前，不需要建立连接。UDP 是不可靠的，可能会产生丢包、粘包等问题，不过它的通讯速度更快。

UDT 是一种新的通讯协议，它是 TCP+UDP 的优化版。它基于 UDP，但是是有连接的可靠通讯。它用了 TCP 那套校验规则，还保持 UDP 的收发速度。

无连接协议有个好处是能多播。比如你的系统里有一大堆电脑，多播能一次性发给网络中的全部电脑。避免了逐个发给每台电脑产生的性能开销。相对应的单播，就得一台一台发。

Websocket 用于跟 Web 浏览器和实时 Web 应用通讯。它在一些双向网络通讯的功能上做了简化。

6. SOP 元件

6.1 说明

Surface 元件，亦称 SOP，在整个元件大家庭中负责处理所有与三维有关的操作。

它包含简单的三维几何体，粒子系统，建筑模型，三维字体等等。因为学习难度稍大以至于 SOP 经常被许多初学者忽视。但是请放心坚实的 SOP 操作知识将给你带来许多意外的惊喜，项目中的灵感，以及提供许多非常有效的解决问题的方法。

许多项目涉及到投影映射，实时三维运动捕捉，建筑表面 LED 空间分布，视频分层播放，而如果没有 SOP 元件这将是非常困难或者不可实现。

TouchDesigner 088 目前所支持的 3D 文件类型有：

- .fbx
- .obj
- .3ds
- .dxf
- .dae

保持元件的渲染流畅度是至关重要的，这将在本书的“优化”章节做进一步的讨论。而当它来自 SOP 时，这一点显得尤为重要。通常的操作是将变换的动画数据给到 Geometry COMP 元件，而不是直接给到一个 SOP 元件。因为 SOP 变换是在 CPU 上执行，并且会将几何体上的每个点都执行一次。而组件级别的变换会将三维几何体或对象视作一个整体，在 GPU 上只运行一次。在 GPU 上的一个单独的操作与在 CPU 上成百上千次的操作相比，前者当然更有效率。

模型上的点数，基元数，顶点数与网格数取决于正在被执行操作模型，但基本原则都是模型的顶点或多边形面数越多，电脑操作时需要的运算能力与分配的 GPU 内存也越大。对于复杂模型 TouchDesigner 有内置工具可以减少多边形面数，但是使用专业的模型工具优化几何体会提升更大的灵活性。

6.2 渲染

许多初学者努力找到一条快速有效的捷径从三维工作流程过渡到二维工作流程。网上有大量的三维工作流程教程可以充分解释三维场景渲染的原理和特点，但是这一章的重点是演示如何将简单的几何对象渲染到 Render TOP。

一个三维场景由三部分组成：

1. 三维几何体（包含材质）
2. 摄像机
3. 照明

打开案例 “Rendering_1.toe”。快速浏览，看到需要被渲染到场景的三个元件：摄像机，灯光与三维几何体，都被 Render TOP 所引用。我们要对它们逐一介绍，然后观察如何将它们连在一起。

让我们从三维几何体开始，因为它是最基础的。三维模型可以是简单的多边形，三维动画人物，建筑模型等等。无论导入还是生成模型，所有的操作都使用 SOP，并且这些 SOP 以 Geometry 组件结束。在 TouchDesigner 中，SOP 自身从不直接被渲染，被渲染的是包含持有渲染标记 SOP 的 Geometry 组件或 Geometry COMP。通过下面两个案例帮助理解这个重要的知识点。

打开 “Rendering_2.toe”。这个项目将一个简单几何体发送给四个有着不同位置的 Geometry COMP。出于案例演示的目的，这个案例使用的是一个 Box SOP，但是它可以是其他更加复杂的模型，这种复杂的模型还可以加上一些动画和交互性。让我们回顾一下知识点：SOP 不能直接渲染，它只在逻辑上将可以容纳迭代的模型载入一次，然后在 Geometry COMP 上渲染。

打开 “Rendering_3.toe”。这与前面的案例不同，它是一个单独的 Geometry COMP 中包含有被渲染的三个不同的模型。记住这点：Geometry COMP 是被渲染的，它将其元件内要被渲染的模型视为一个组。现在看起来这并不是一个很重要的行为，但是随着项目越来越复杂，节约资源将会变得非常重要。

到现在为止，各种几何体都没有材质。材质可以将三维场景变得有趣。使用水泥做的方块与普通盒子之间的区别就是材质。纹理可以被应用于两个级别中：SOP 级别与 Component 级别。第一种方法是使用 Material SOP，第二种方法是在 Geometry COMP 的 “Render” 面板引用材质。在案例 “Rendering_4.toe”，虽然材质看起来是一样，但是它们使用不同的纹理映射方法。

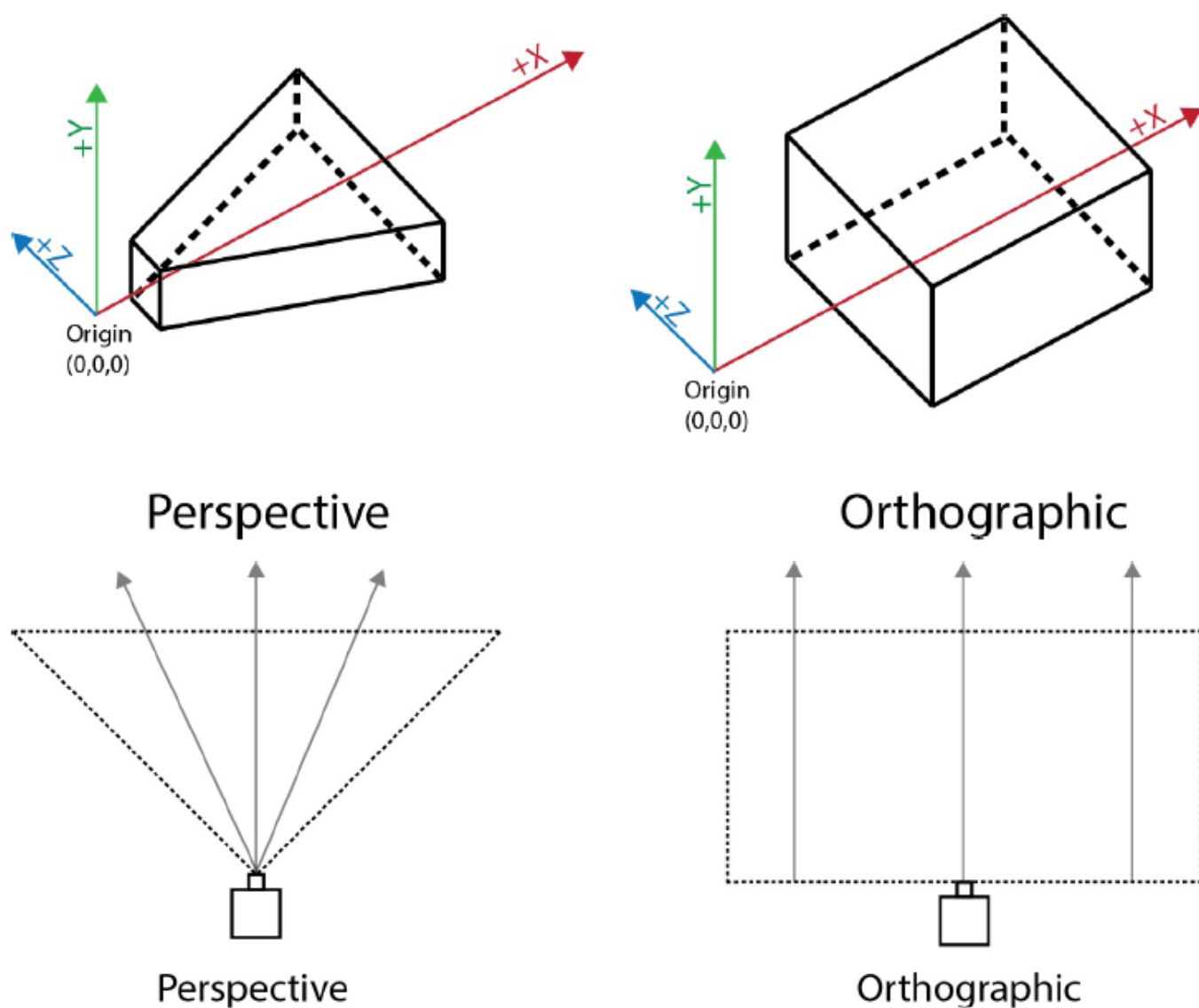
现在，材质被添加到以上的模型中了，现在让我们快速讨论一下 UV 贴图。打开 “Rendering_5.toe”。这个案例与上一个相同的案例，但相同的纹理其效果看起来完全不同。这是因为 UV 贴图坐标被改变。

三维场景的第二个部分是灯光。就如现实生活一样，三维场景需要照明。打开案例 “Rendering_6.toe”。在这个案例我们看到一个简单的盒子被渲染，但是在 Rendering TOP 中却没有可见的部分。这是为了通过将光线的明暗（dimness）被设置为 0 以说明照明的重要性，并且它是如何经常被忽视的。

接下来三个案例 “Rendering_7.toe” , “Rendering_8.toe” , “Rendering_9.toe” 是相同场景中使用不同的灯光。在每一个项目中, Light COMP 从不同的角度对 Box SOP 进行照明。

渲染三维场景的最后部分是摄像机。摄像机是眼睛和视角。摄像机看到什么就会渲染什么。打开案例 “Rendering_10.toe” 。整个场景中的运动都来源于摄像机的运动。通常, 当我们 将摄像机放置好之后, 就把它忘记了。这会导致一个静态, 没有活力的枯燥场景。不要害怕 像一个摄影师一样思考, 尝试改变摄像机位置, 焦距与摄像机运动。

有两种摄像机类型: 透视摄像机与正交摄像机。



透视摄像机不用多解释, 它的工作原理如人眼。透视摄像机由透视点与视锥用于决定摄像机看到的画面将被如何呈现。对于透视摄像机, 整个的三维场景都像是集中到一个点。透视校正, 亦称 “近大远小”, 被应用于场景中的所有对象, 即对象距离摄像机越远看起来越小。这种摄像机最经常被应用于模型人眼。在案例文件 “Camera_1.toe” 有这种类型的摄像机。

这个案例重点展示透视效果。有两个相同的立方体。在场景中放置较远的立方体显示较小，如现实中看到的一样。这是透视摄像机功能的最简单演示案例。

正交摄像机与透视摄像机有很大不同。那些熟悉的计算机辅助设计（CAD）软件的朋友，在这之前可能已经用过正交摄像机。正交摄像机的核心原理是没有透视点。三维场景中的所有灯光都不会像在透视摄像机中那样有一个唯一的透视点。对象不会在它们的 Z 轴上发生变形，意味着无论它与摄像机的距离如何改变，它都不可能看上去比场景中的其他对象更大或更小。

打开案例“Camera_2.toe”，这个场景与“Camera_1.toe”完全一样，但是最大的不同是渲染画面，无论两个立方体在 Z 空间如何定位，它们都并排显示。第一次接触会比较难以理解。那么就跟随下面两种思维方式：

第一种，一个三维世界在被渲染前被挤压到一个单独的平面上。想象一下超级马里奥游戏其实是一个三维场景，在渲染时使用正交摄像机。不管坏人与平台在屏幕的边缘还是中心，它们没有透视关系，总是拥有相同的尺寸与形状。

第二种，思考在 CAD 程序中设计一个建筑的蓝图。它是在二维平面上表示一个三维对象。无论蓝图显示在屏幕的哪个位置，在边缘测量的一个单位长度与在屏幕中间测量的一个单位的长度完全相同。摄像机位置的任何变化都不会变形正在渲染的画面。

这并不是说当使用正交摄像机时景深就变得无关。相反，当几何体在场景中发生层叠时，景深变得非常必要。

摄像机在投影映射项目中同样重要。投影映射将在以后的案例中探讨，但现在，关键在于理解摄像机的另一个重要角色。在投影映射中其目标是将现实对象变成三维显示表面。要做到这一点，一个对象模型需要被导入到 TouchDesigner，使用各种资源来为它添加纹理。这个纹理对象接下来需要被渲染。Camera COMP 用来模拟现实中的投影机。收集尽可能多的投影机的特征信息以及它的镜头，Camera COMP 可以在 TouchDesigner 中精确模拟投影机的视野设置（FOV）。这些视野设置将被渲染，输出，校准与 lined up 现实对象，这就是投影映射的基础。

7 COMP 元件

7.1 介绍

有三种类型的 Component 元件亦称 COMP，各有用途：

- **Object** 组件创建、照明与观察三维场景。
- **Panel** 组件创建 UI 组件如按钮、滑块与窗口面板。
- **Other** 组件创建关键帧动画，复制元件，创建输出窗口。

组件元件通常与其他元件组合使用。“Object” 组件通过多种组合用于创建与渲染 SOP 和三维场景。“Panel” 组件用于创建 UI 和其他的元件 (例如配合 container 等) 制作操控输出端的 UI 操作面板。“Other” 组件用于完成各种任务，如关键帧动画，动态复制元件，在多输出端显示不同窗口等等。

一个有趣的事实是如此庞大的 TouchDesigner 由本身的内部组件组成。理解这一点非常有助于对 TouchDesigner 的细节把握，以及理解 TouchDesigner 在项目中的运作。例如所有的面板组件都是由其他元件组成。创建一个 Button COMP，进入它的网络，它的背景是由一个 Text TOP 创建，它的 开/关值是被一个 Panel CHOP 生成。相似的，TouchDesigner 的所有 UI 都是被创建并存储于所有项目根目录的“ui”容器内。甚至菜单与对话框，如 MIDI Mapper Dialog 与 Variable Dialog 都是由 TouchDesigner 的其他组件创建。

7.2 Window COMP

几乎每一个项目都会使用 Window COMP 创建一个新的窗口显示元件的内容。无论使用 Window COMP 创建全屏输出还是创建更为传统的窗口应用程序，都有许多选项要修改。因为每个项目的独特性，没有“最佳设置”可以依赖。去 Wiki 论坛上花费一些时间查看这些参数，在每种情况中实验找到它们的最佳设定。

打开案例“Window.toe”。这个案例介绍一个非常有用的练习，以及一些 Window COMP 的功能。这是使用 Container COMP 作为 Window COMP 的输出目标的最佳练习。这使得即使在 Perform Mode，TOP 中显示的内容也可以在不同输出端之间任意拖动。如果我们仅仅使用 container，我们没法去设定窗口的大小和位置，以及选择输出端口，而这些通过 windowCOMP 就能完成。

这个案例中的其他方法都是比较简单的。利用 Open Window 按钮打开图片，然后利用 Change Monitor 将图片在不同输出端之间切换。MonitorsDAT 是个很有意思的元件，在这个案例中用以动态检查有多少显示器被连接。

7.3 用户界面组件

组件元件可以在 TouchDesigner 中创建用户界面这使得它显得非常重要。具体而言，Panel 组件提供这些功能。许多用户界面将在以后的例子中创建，所以这一部分只分析几个基础案例。

三个用处最大的 Panel COMP：

- Slider COMP
- Button COMP
- Container COMP

前两个的功能与其他应用程序中的滑块与按钮功能一样，但是可以依照需求不同而修改。按钮可以被编程为开关，单选，或者单次点击事件。滑块可以被定义为单轴滑块或全方向滑块（例如一个 UV 的二维区间）

然而，Container COMP 就仅仅是一个存放其他元件的容器，没有很特别的地方。

打开“UI.toe”。在这个简单 UI 的案例。由下向上观察这个网络结构，有 2 个 Button COMP 与 5 个 Slider COMP。这些是被用来创建 UI 功能的组件。这些元素的父节点用以将按钮与滑块分开并分别成组。注意即使是“container1”和“container2”是被激活的，UI 元素是可用的，但是“container1”和“container2”都没有输出，在它们内部网络也没有元件。

类似的，当“container1”和“container2”由“container3”组合起来时你会看到同样的结果。这是因为 Container COMP 有在视图中显示子容器的功能。Container COMP 可以通过组合多个小容器来创建复杂界面。

8. MAT 元件

8.1 简介

Material 元件亦称 MAT，其主要功能是用于为三维几何体添加材质与着色。

在进行材质或着色工作时若对计算机图形学与渲染有深入的了解会有很大帮助。如果没有这方面知识，大部分用户将只会使用 Phong MAT 与 Point Sprite MAT 的基础设置。

材质还要涉及到纹理与 UV 贴图，最好是使用特定的模型工具作提前处理。在 TouchDesigner 中可以导入模型添加纹理。

有许多修改纹理的工具，如 Texture SOP，但是对于复杂的操作还是建议使用特定的软件处理。

8.2 Phong、GLSL、与 Point Sprite 材质

以下是三种最常使用的 MAT 元件：

- Phong MAT
- GLSL MAT
- Point Sprite MAT

这是三种不同类型的元件，可以满足大部分的着色与材质需求。

Phong MAT 是最常用的材质元件。它负责将纹理映射到三维几何体。它支持多种贴图，如颜色、凹凸、高光、漫反射等等。Phong MAT 能与环境、漫反射，镜面反射，散射和基本平面光照明（无深度信息）混合。打开“Phong.toe”。在这个项目，有两个非常简单的 Phong MAT 例子。第一个使用纹理的透明通道创建一个透明盒子。第二个忽略场景中的照明，设置使用（1,1,1）的 **Emit** 灯光将对象完全照亮。在许多案例中会经常看到 Phong MAT 的应用。

GLSL MAT 用来使用 OpenGL 着色语言（简称 GLSL）创建自定义材质。GLSL 是非常强大的编程语言，它可以创建运算快速的而且非常复杂的纹理。它给程序员大量的操作图形接口，而不用直接使用汇编语言。第一次学习会感觉稍难，但是网上有海量的 GLSL 着色案例，在 TouchDesigner 论坛的“组件分享”区也有大量的案例。

Point Sprite MAT 为粒子系统的点匹配 sprite 。从它的名字理解它的用途，将二维图片（ 或一个 sprite ）替换三维空间中的每一个点。Sprite 总是朝向摄像机，根据 Z 轴深度进行缩放。查看案例 “Point_Sprite.toe” ，在 TouchDesigner 中若不使用 Point Sprite 创建一个类似的效果，不仅仅会是一个相当杂乱的网络，也不知道要添加多少个 Transform TOP 与 Composite TOP，另外它们会占用大量的系统资源。通过使用粒子系统 与 Point Sprite，网络结构易于查看，并且不需要大量系统资源。

8.3 UV 贴图

对于复杂三维几何体，UV 映射是非常重要的工作。作为三维建模中的另一部分，在专业的三维建模软件中，创建与操作 UV 贴图是比较简单的。

UV 映射允许设计师和艺术家为三维几何创建有趣的动态或静态纹理。它架起了动态与静态图形的二维世界与三维几何世界的桥梁。

UV 映射有三个步骤。第一步，将三维对象展开到二维平面。这个展开的贴图称为 UV 贴图。可以把它看作是一幅地图，因为它非常像其他类型的地图，它是依据三维模型按比例精确创建出来的二维参考。如街道地图或世界地图，将三维世界放到二维平面。

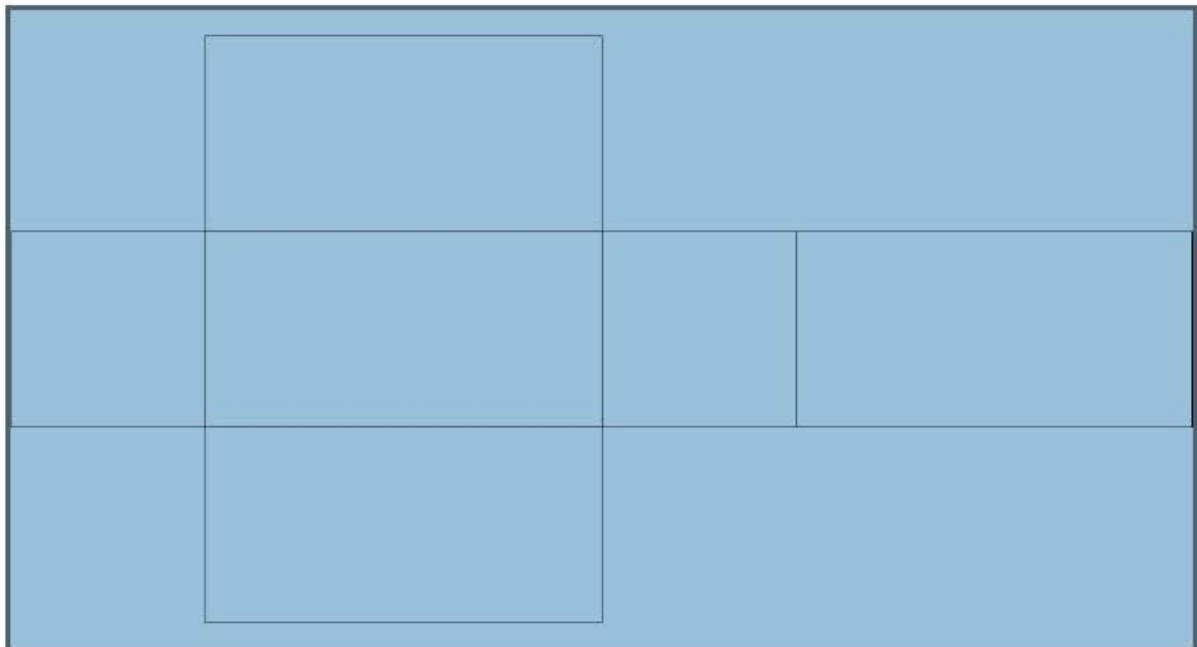
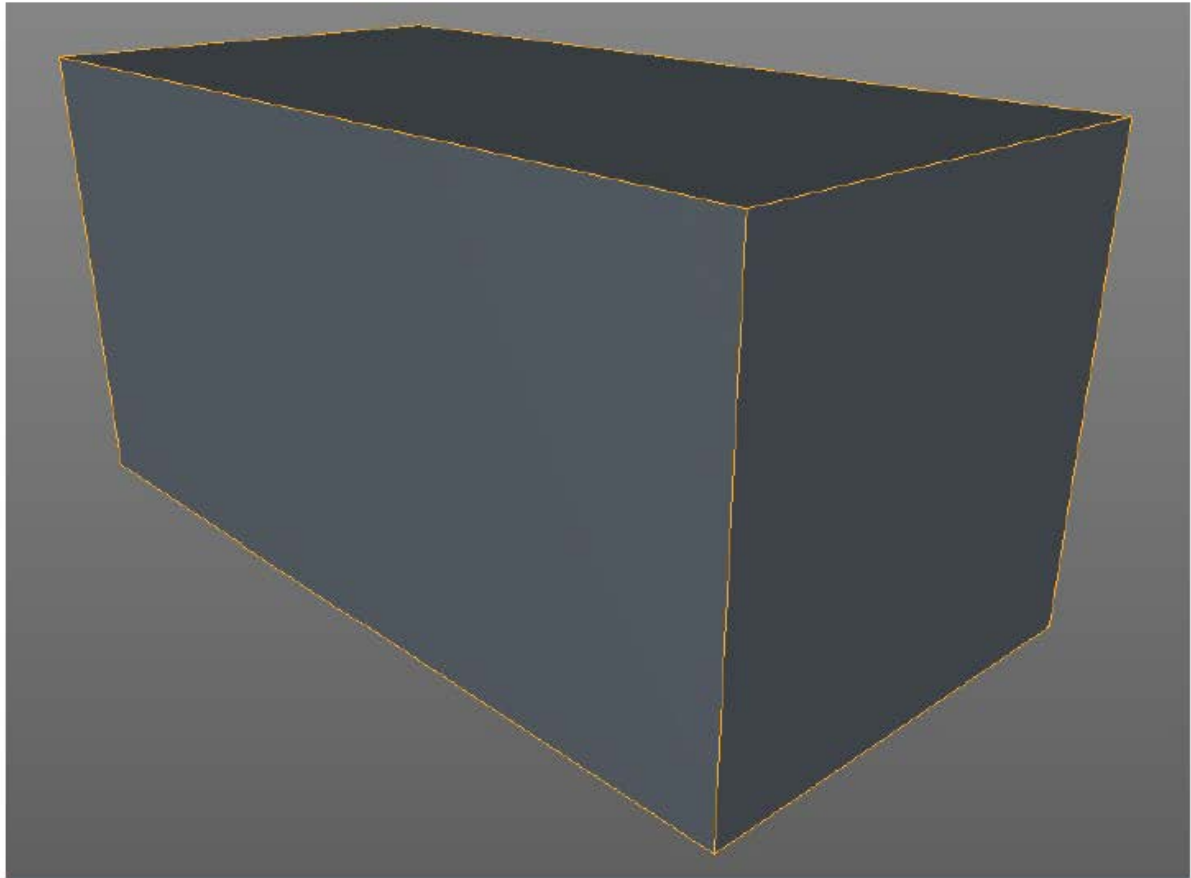
第二步是创建纹理。二维 UV 贴图被艺术家和设计师用来在合成软件中创建动态或静态的纹理。UV 贴图的优点是可以将纹理精确的映射到几何体上。

第三步是纹理在三维几何上的应用。这取决于所使用的软件而有所不同。

这三步合起来称为 UV 映射。

第三步常在 TouchDesigner 中操作。只要三维几何从它的建模软件中正确导出，它将包含指引其他应用程序应用 UV 贴图的坐标。在这种情况下，通过 Movie In TOP 载入纹理，添加 Phong MAT 到几何体。如果需要改变 UV 贴图的应用方式，可以使用 Texture SOP。下面是一个简单三维盒子与它的 UV 贴图案例。

Introduction To TouchDesigner



9 Python

9.1 介绍

Python 脚本是 TouchDesigner 0.88 中最强大的功能之一。它能做非常复杂的操作，如迭代运算大量数据集合，自带的大量 Web 通信 API，能以一种超乎寻常的速度去控制和改变元件参数等等。

有一点需要注意，TouchDesigner 088 使用 Python 3.3.1。Python 3 在许多地方与 Python 2.7 不同，但是大部分问题都可以通过谷歌搜索得到解决。许多问题可能已经被提过，其解决方法可能只是添加一组小括号。

例如，在 Python 2.7，Print 函数可以不添加小括号，但在 Python 3 中会将抛出一个错误。下面的案例是在 Python 2.7 中使用 Print 方法。

```
print 'text'
```

在 Python 3 中使用对应的 Print 方法，小括号是必需的：

```
print ('text')
```

Python 中一个令人难以置信的功能是除了它的代码可读性外，它还有许多的外部库，其中一部分可以直接连接到最活跃的网站，如 Facebook、Twitter、Foursquare 等等。这类开放式实时数据驱动项目作为可视化项目和为客户演示的实时数据。

在后面的案例中会有相当一部分涉及到 Python 脚本语言，虽然不强制这么做，但还是强烈建议一定要花一些时间学习 Python 的入门教程。有许多 Python 的学习资源，其中一些是做游戏，它们中的大多数很容易找到并下载下来。每天花费 10-15 分钟，坚持几个星期，可以大幅提升 Python 的基础知识，以及拥有足以编写 TouchDesigner 脚本的能力。

9.2 Textport

在 TouchDesigner 中的 Textport 对于脚本使用扮演着几个重要的角色。有两种打开 Textport 的方法，第一种使用快捷键 “Alt+T”，第二种在 TouchDesigner 窗口的顶部 “Dialogs” 菜单中选择 “Textport”。

第一种方法是自带的编辑器，就像我们在 Python 自带的 IDLE shell 编辑环境一样。例如，打开 Textport，输入下面代码，按回车确认。

```
print(2+2+2+2)
```

在 Python 中运行这句代码会打印出等式结果为 8，在 Textport 中按回车后同样会显示等式结果。这是因为 Textport 作为 Python 解释器运行。键入 Python 代码之后，它将会被执行，并返回结果。

对于在 TouchDesigner 077 中运行的脚本语言 tscript 可以作相同的运算。注意当 Textport 被打开时，第一次打印出来的信息是：

```
python >>
```

这是因为在 TouchDesigner 088，Textport 的默认解释器为 Python。要在 Textport 中运行 tscript，需要将 Textport 的模式从 Python 改为 tscript。点击 Textport 左上角的 Python Logo 标志，它将会变成字母“T”，在 Textport 的下一行将打印：

```
tscript ->
```

这就说明 Textport 目前处于 tscript 模式。在 Textport 中输入下面代码进行验证：

```
echo Hello!
```

与 Python 的 Print 函数相似，上面的 tscript 代码会在 Textport 中打印“Hello!”。

Textport 的另一个重要功能是作为 Python 的调试器。打开“Textport_1.toe”。

这一简单例子主要演示当运行 Python 脚本时如何结合 Textport 一起使用。打开 Textport，点击“Good Python”按钮，运行脚本，会在 Textport 中打印如下信息：

```
this will not error
```

现在点击“Bad Python”按钮，Textport 显示一些非常不同的信息：

```
File "/project1/chopexec1", line 11
```

```
Print(this will error)
```

```
^
```

```
SyntaxError: Invalid syntax
```

这是一个 Python 错误，主要因为一些代码被解释为无效的。学习阅读这些错误可以加速调试大型 Python 脚本。让我们检查这个错误的各个不同部分。

第一行指定错误出现于网络中的确切位置，以及哪一行脚本是被 Python 解释器认为是无效的。

File "/project1/chopexec1", line 11

在这个案例，错误的元件是“project1”组件内的“chopexe1”元件，Python 在脚本的第 11 行停止解释。

下面，Textport 打印的错误代码行：

Print(this will error)

多数时候，拼写错误和大小写错误可以轻易的通过这两行发现。在这种情况下，很明显要打印的字符串不在封闭的引号中。知道错误原因和出错位置，意味着问题可以很快得到解决。再来看看错误的最后一行：

SyntaxError: Invalid syntax

错误的最后一行是 Python 遇到的错误类型。关于不同错误类型的更详细信息可以在 Python 3.3 的离线文档中的“Errors and Exceptions”章节找到。“SyntaxError”是很常见的错误，会导致 Python 解释器遇到不正确的 Python 语法代码。如上所述，上面的代码是因为要打印的字符串丢失了引号。

在 Textport 中打印信息是脚本输出的一种调试方法。在大型项目中脚本随处可见，而且它们中有大都在后台执行任务，例如影片的预加载与卸载，它们的运行结果都很难被观察到。多数时候，脚本在不同网络中是并行运行的，这就很难观察到脚本的运行时间是否正常，以及判断它们动作发生的先后顺序。

在运行脚本时通过打印方法，许多零碎的信息会被打印出来，不易于分析脚本。这些信息片段可以简单作为被运行脚本的路径，或作为被改变值或改变的详细说明。

打开“Textport_2.toe”，在这个案例，有两个依次间隔运行的脚本序列。单击“Visual Sequence”按钮，通过将序列中按钮设置为 On 启动每个脚本的运行。这是为了更清楚看到这个序列的运行情况。那么如何从其他网络监视这个过程呢？

打开 Textport，单击“Textport Sequence”按钮。不同于第一个序列，这个序列把在 Textport 上打印信息作为每个脚本的运行。在网络中没有显示改变状态的按钮，但是有一些新的功能。第一，在项目中的任何位置都可以监视这些脚本。第二，在项目中的其他位置脚本都可以对比这些序列的运行时间。第三，这可能是最有用的，暂停项目的运行并将最近序列事件写入历史记录。

当调试拥有复杂逻辑的系统时，这段历史记录就变得绝对有价值。在一个拥有 30 个网络 and 300 个脚本的项目中，当一系列动作失败却没有任何 Python 错误抛出时，没有一个有序的日志记录，就根本不可能跟踪调试。由于脚本变得越来越复杂，在脚本中轻易创建更多的这样伪检查点，如“Y 脚本的 X 部分正在运行中”。

9.3 规范

在 Python 中有一些约定俗成的东西在工作时应该使用。

关于 Python 语言的一些有趣东西构建在它可读与易用的思想上。在 Textport 或 Python 解释器中输入下面代码，可以看到 Python 的一个复活节彩蛋：

import this

Python 返回一首诗，名为“The Zen of Python”，由 Tim Peters 创作。下面是原文：

Python 之禅

赖勇浩翻译

优美胜于丑陋（Python 以编写优美的代码为目标）

明了胜于晦涩（优美的代码应当是明了的，命名规范，风格相似）

简洁胜于复杂（优美的代码应当是简洁的，不要有复杂的内部实现）

复杂胜于凌乱（如果复杂不可避免，那代码间也不能有难懂的关系，要保持接口简洁）

扁平胜于嵌套（优美的代码应当是扁平的，不能有太多的嵌套）

间隔胜于紧凑（优美的代码有适当的间隔，不要奢望一行代码解决问题）

可读性很重要（优美的代码是可读的）

即便假借特例的实用性之名，也不可违背这些规则（这些规则至高无上）

不要包容所有错误，除非你确定需要这样做（精准地捕获异常，不写 `except:pass` 风格的代码）

*当存在多种可能，不要尝试去猜测
而是尽量找一种，最好是唯一一种明显的解决方案（如果不确定，就用穷举法）
虽然这并不容易，因为你不是 Python 之父（这里的 Dutch 是指 Guido）
做也许好过不做，但不假思索就动手还不如不做（动手之前要细思量）
如果你无法向人描述你的方案，那肯定不是一个好方案；反之亦然（方案测评标准）
命名空间是一种绝妙的理念，我们应当多加利用（倡导与号召）*

这首诗是许多 Python 开发者的座右铭。它的字里行间传达着 Python 的理想和开发的规范。

思考这一行：

'Explicit is better than implicit'

明了胜于晦涩（优美的代码应当是明了的，命名规范，风格相似）

这句话可以用到经常匆忙使用的事情上：命名。在 Python 与 TouchDesigner 中有许多不同地方都有对象之间通过名称互相引用。每一个开发者都要命名变量、方法，元件、网络等等。如果没有对元件仔细命名，不可能知道每个元件的方法。类似的，本章节前面的部分已经证明，如果没有仔细命名变量名，阅读脚本会变成一件非常枯燥的任务。在 TouchDesigner 中当命名元件或变量时有两种格式：

第一种，每个单词之间使用下划线分割。如：

```
final_comp  
stop_button  
time_now
```

下划线分割使得名字容易阅读并快速理解。但有一些人不喜欢使用下划线，而是在不同的单词之间使用大写字母来区分单词。如：

```
finalComp  
stopButton  
timeNow
```

这两种方法都可以明确传达开发者的原始想法，促进合作。

9.4 注释

对于 Python 初学者，注释应该作为一种练习，不能被忽视。它很简单，只需要几秒钟来说明那几行代码的作用。在许多情况下它非常方便：

- 程序员之间分享代码与项目
- 阅读以前的项目代码
- 重用函数

当脚本比较短时，注释看着比较琐碎，但是它应该从学习第一天开始积累。看下面的代码：

```
a = 10
b = op('value')[0]
op('op12').par.rx = b
c = str(op('numbers')[0])
d = 'testing'
e = 'something'
op('lines').par.text = d + e + c
```

这个脚本难以理解有多种原因，其中最大原因是它的作用不明确。一个快速增加脚本阅读效率的方法就是添加注释。让我们为上面的代码添加注释之后再来阅读：

```
#Set initial variable
#get the value from slider input
a = 10
b = op('value')[0]
#assign the slider value to video input rotation
op('op12').par.rx = b
#take numeric input from computer keypad
c = str(op('numbers')[0])
d = 'You'
e = 'pressed'
#create a sentence from variables above
#add it to a Text TOP to display
op('lines').par.text = d + e + c
```

不用花费时间思考创建有具体含义的变量与元件名，上面的代码仍然非常容易阅读。即使脱离上下文，这个脚本的功能也非常显而易见。

这种简单的添加注释可以使开发者之间的工作更轻松愉快。

9.5 划分

项目变得越来越复杂时，脚本也慢慢得变得越来越长。在这时 TD 将需要花费更多时间搜索代码，甚至于超过运行代码的时间。基于这些原因，脚本的划分对所有项目都非常重要。

这意味着我们需要运用不同的模块和技术去改善工作流程，而随之而来更大的好处是它们会消除项目合作间的痛苦。其他优点如：

- 易于长期维护脚本
- 花更少时间向同事解释脚本
- 增加程序的重用性
- 更快速的进行简单编辑和代码维护

打开 “Scripting_1.toe”，在这个案例中，有 10 个 Movie In TOP 并执行各自的动作。首先，将每一个视频卸载。然后，为第一个 TOP 匹配一个文件路径。最后，为每个视频的播放作预载。因为这是一个 Python 案例，所有动作都使用 Python 脚本执行。下面查看 “movies” Text DAT。

快速浏览一下脚本：出于案例演示需要，这个脚本没有使用循环遍历所有的操作元件。虽然只执行几个单独的动作，但这足以体会到操作一个又长又复杂的脚本是什么感觉。

在 “movies” Text DAT 上点击右键，选择 “Run Script”，所有的动作将依序运行。这个脚本已经添加了注释，但是仔细阅读代码仍会让你感到困惑。如果要在代码中编辑其中的某个值，不得不在长长的动作列表中查找。同事们将花费更多时间才能弄清脚本的作用。而在将来要重用脚本中的代码片段时，需要手动查找并提取。如何才能提升这个过程效率呢？

打开 “Scripting_2.toe”，这个案例使用上一案例的代码，但是将我们的动作按类独立分割在不同的 Text DAT 中。

即使不查看每个脚本，也可以快速的知道各个元件的功能：一个卸载视频，一个改变路径，一个预载视频。

在每一类脚本的结尾有一行代码将按序运行每一个过程的脚本。该行代码使用 Run 类。

op('preload').run()

这可以很轻易的编辑任意一个动作或参数值，并进行跟踪。向同事共享这个组件时，他们第一眼就可以看明白每个脚本的作用。如果有人需要启动时预载一组 Movies In TOP，也可以快速的从这个项目中提取脚本。

但这样的划分方法难以管理超过 500 行的脚本，分成更小的脚本易于管理与分享。在这种情况下，有第三种方法来控制多个元件的运行。

打开 “Scripting_3.toe”，这个案例使用到 Python 函数。一个 Python 函数是一小段代码，可以被作为一系列动作进行调用。在 “actions” Text DAT 中，定义一个函数，它包含每个 Movie In TOP 需要被执行的一系列动作。在 “set_movie_tops” Text DAT 中而同样的动作不是被一遍又一遍地重复输入，而是每一个 Movie In TOP 都调用执行此函数。

虽然这一系列动作有些牵强，但是它的主旨是明确的：使 Python 脚本易于维护，在工作流程中易于使用，可重复利用，且方便管理。

9.6 外部模块

在 TouchDesigner 中集成的 Python 有尚未完全发挥的能力那就是它可以导入与使用 Python 的第三方库。

这包括使用当下比较流行的 Requests 库或 Beautiful Soup 库。导入库的过程非常简单：

1. 安装 64 位的 Python，在写此文章时的最新版本是 3.5。
2. 使用 “Pip” 安装要用到的外部模块。
3. 打开 TouchDesigner， “Edit” 菜单下选择 “Preferences”。
4. 在 “General” 下找到 “Python 64-bit Module Path” 参数，添加 “site-packages” 目录的路径。可以在 Python 安装目录中的 “Lib” 目录下找到。

5. 进入 TouchDesigner , 创建一个 Text DAT , 使用标准 “import” 命令进行测试。这个方法常用于外部模块的版本测试 , 用户经常遇到的问题是使用 “Pip” 安装外部模块虽然安装成功但版本却不是期望的。

9.7 哪里学习 Python

下面是一些在线免费学习 Python 的资源（注：有一些非常好的教程是使用 Python2.7 , 所以也一并包含在内）

CodeAcademy <http://www.codecademy.com/tracks/python>

Khan Academy <https://www.khanacademy.org/science/computer-science>

Coursera <https://www.coursera.org/course/interactivepython>

Google Developers <https://developers.google.com/edu/python/?csw=1>

Learn Python The Hard Way <http://learnpythonthehardway.org/>

10 如何部署内容输出以及 Perform (演出) 模式

10.1 Perform (演出) 模式

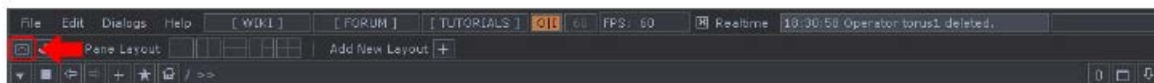
在部署项目时应当尽可能的使用 Perform 模式。Perform 模式使用的前提是当一个项目在交付时，不需要网络编辑器，它将处于不可编程的状态。渲染网络编辑器会占用大量资源，尤其当有许多可视化元件、显示 CHOP 通道，TOP 预览，DAT 表格等等。

因此，Perform 模式的存在使得计算机可以专注于渲染内容，不必额外的渲染网络编辑器。在 Perform 窗口中所有涉及创建最终效果的元件仍然被渲染与激活，包含外部数据的输入与输出。唯一被停止渲染的是网络编译器。

在组件章节中曾推荐 Container COMP 作为 Window COMP 的来源。这个建议同样适用于 Perform 模式。

打开 “Perform_mode.toe”，进入这个案例，“container1” Container COMP 作为 Perform 模式的来源。按 “F1” 进入 Perform 模式，容器的内容会显示在新窗口。按 “ESC” 退出 Perform 模式，返回网络编辑器。

点击这个 UI 按钮同样可以进入 Perform 模式。



10.2 网络编辑模式下的性能消耗

因需求各异，有时必须在现场演出时进行编程。如果是这样的话，有一些操作可以帮助优化管理网络编辑器对系统性能的影响。首先是关闭所有元件的可视化状态以节省资源消耗。看下面的图片：



下一步是在网络编辑器中只使用一个窗体。越多的窗体被显示，需要被渲染的数据量越大。

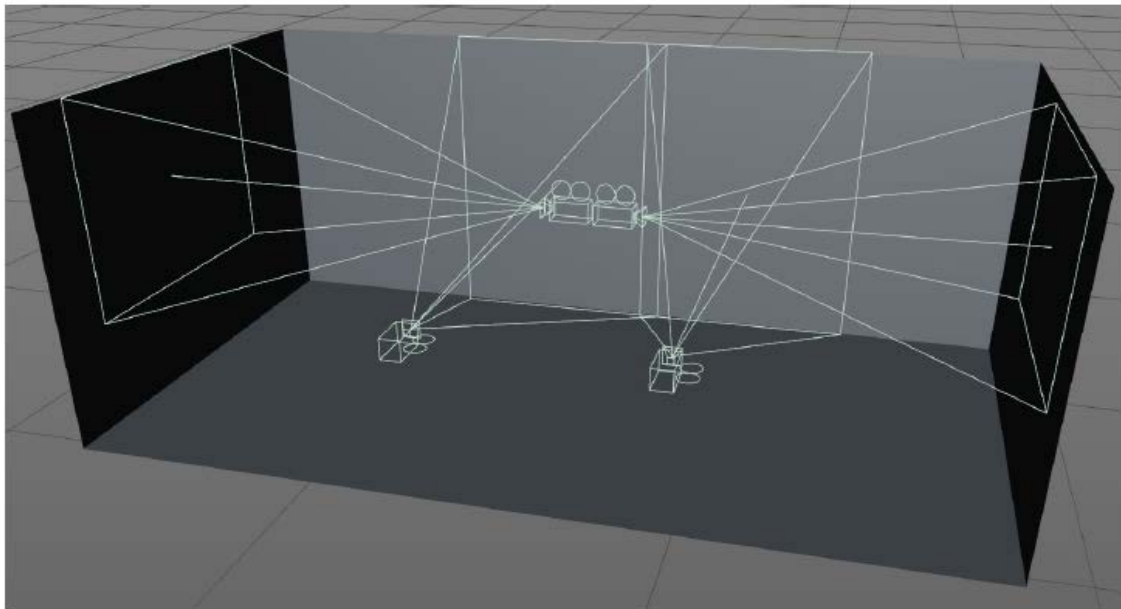
最后，应该避免在大型网络中执行进入或跳出操作。这种行为会导致丢帧现象，原因是在显示之前 TouchDesigner 必须渲染网络中的所有元件。

10.3 创建和制作输出栅格

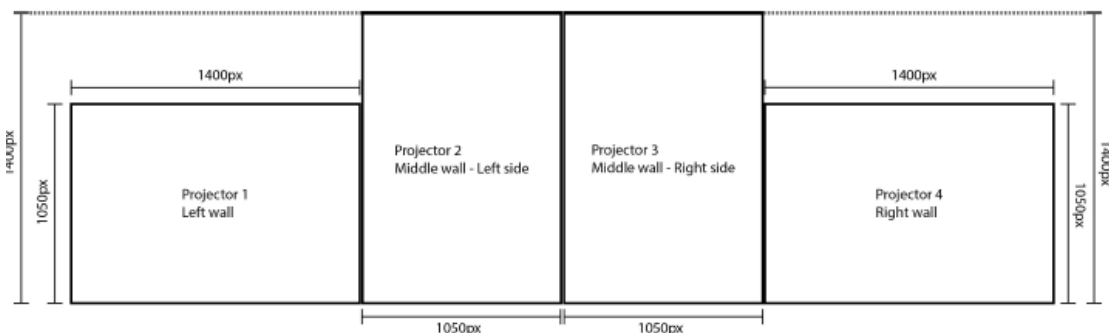
要得到最佳的演出效果，通常的规则是一次只使用一个 Window COMP。这并不适用于设计，但当它涉及到演出调度超过一个窗口时将大大降低系统性能。所以在不同的位置或旋转角度有多个输出时该怎么做？

答案是为输出创建 raster（栅格），创建一个包含全部的现实输出的单一窗口。

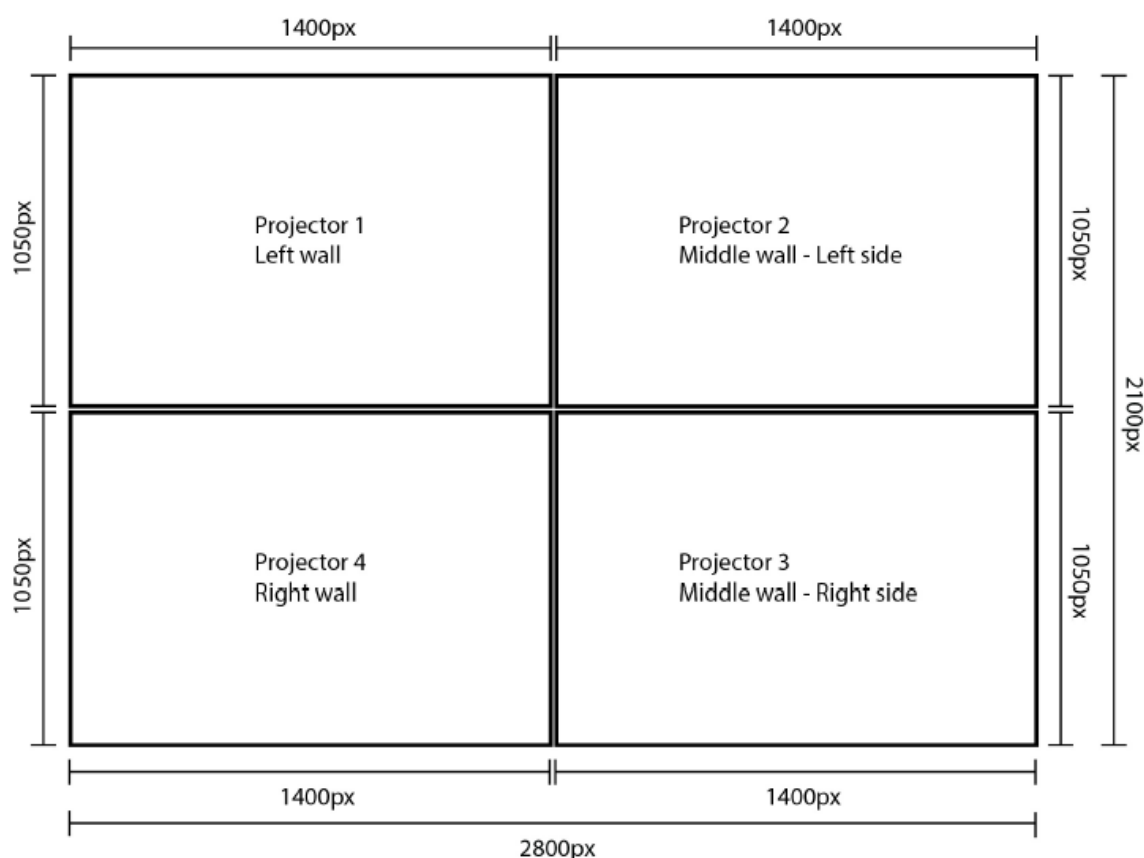
用一个简单的例子作说明。在这个示例场景中，有四台 SXGA+投影机，每一台的分辨率都是 1400*1050。在现实世界中的设置，有两台投影机分别水平投射在两侧的墙上，有两台投影机垂直排列并进行边缘融合投在中心墙上。下图展示要进行的设置：



这种设置并不是特别复杂，但知道如何有效的处理是非常重要的。让我们把这些设置在平面上布局显示。



初学者的第一反应是因为有四个输出所以需要使用 4 个 Window COMP，其中两个需要旋转。现在面临的挑战是只创建一个 raster（栅格布局）如何使得布局最有效？在这种情况下，因为四个输出的分辨率是一样的，一个简单的解决方案是使用 2*2 的网格。



在上图中，所有输出被放置在单一的 raster（栅格布局）中。这种设置可以有效的使用一个 2800*2100 像素的 Window COMP。此时，在 Windows 系统中 NVIDIA 与 AMD 的控制面板应该使用相同的监视器设置，使得它被正确地连接在显卡上输出。

下一步是在 TouchDesigner 中准备这个 raster。打开 “Raster.toe”，有一些地方需要在开始时就注意到。为了演示这个案例，创建一些非常基础的虚拟内容，它们用来表示将来投影出去的真实内容。1400*1500 像素的 “Content1” 是左侧投影机的内容。2100*1400 像素的 “Content2” 是中间投影机的内容。1400*1050 像素的 “Content3” 是右侧投影机的内容。它们都被放置在名为 canvas 的 Container COMP 内。

在 “canvas” 容器，信号路径从左至右从上至下，像瀑布一样。首先是创建用于合成内容进去的空白 raster。网络左上角的 Constant TOP 设置分辨率为 2800*2100 作为空白 raster。根据上图，使用 overTOP 在 raster 中指定第一块内容即 1 号投影机内容的位置，通过 “Translate” 参数进行微调，1 号投影机设置完成。

中间的两台投影机要进行边缘融合。为了对原始内容做处理，把它们放入名为“crop”的容器内。将操作封装起来，使界面整洁也便于查找。进入“crop”内部，有三块主要的操作。首先将整块内容切分为两块，使得每台投影机投射一半的图像。投影机是垂直安装，但是在 raster 中却是水平放置，因而要用 Flip TOP 的“Flop”参数用来旋转画布。因 Flip TOP 的设置基于硬件不同而不同，故尝试不同的 Flip 与 Flop 设置得到正确的画布方向。

边注：许多初学者在旋转整个画布时会遇到这样的麻烦，他们马上会想到使用 Transform TOP，但一定要注意的是 Transform TOP 旋转的是画布内的像素。这也是为什么使用 Flip TOP 中“Flop”参数的原因，它旋转的是整个画布。因为这个案例不是致力于边缘融合，因而“edge_blend”容器只是一个占位符，来用创建边缘融合的视觉效果。

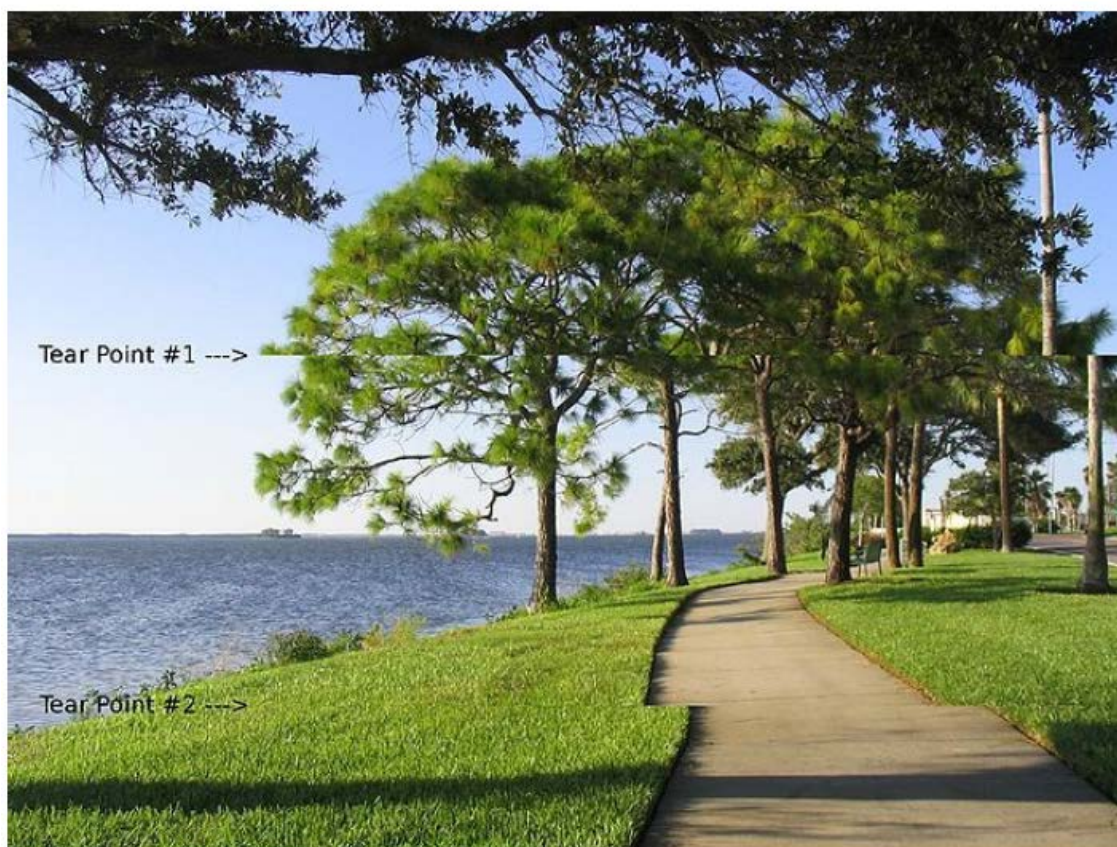
当裁剪，旋转，融合完成之后，准备将两台投影机的输出合成到 raster。与之前同样的方法，Over TOP 中的“Translate”参数定位两块内容的位置。现在 2 号与 3 号投影机设置完成。

最后的投影机与 1 号投影机操作一样，使用 Over TOP 将最后一块添加进去。

在前面章节提到，最好使用 Container COMP 代替 TOP 作为 Window COMP 的来源。在这个项目中，有一个 2800*2100 分辨率的容器来容纳整个 raster。Window COMP 的“Operator”设置为“final”容器，“Borders”设置为关闭，窗口尺寸设置为 2800*2100。做完这些，这个项目输出画面给 4 台投影机的准备工作完成。

10.4 Displays, Tearing, and Stuttering

本书不涉及到太多对于硬件的讨论，但是在进行多屏显示时有一点需要引起你的注意。最重要的一条是显示设备的硬件配置必须完全相同。信号流中的任何差异都可能导致画面撕裂。如下图，画面撕裂效果与这个相似，注意看画面有水平的错位：



图片来源：Wikipedia

在显示刷新图像与显卡渲染图像不同步时会发生撕裂。这样的画面，其中一部分来自于上一帧，而一部分来自于下一帧。对于缓慢移动的内容，有时难以发现这一点，但是在画面中一旦有了运动，撕裂效果会分散人的注意力。

撕裂是一个复杂的问题，但是采取一些预防措施可以避免。首先是使用专业的显示，如 NVIDIA Quadro 系列显卡。大部分公司只会在他们顶级专业显卡上保证不会出现撕裂的效果。

第二要始终确保显示设备的设置相同。如果安装有三个刷新频率为 60Hz 的输出，一个刷新频率为 59Hz 的输出，会有可能发生撕裂现象。如果安装两台 1080P 的投影机与两台 SXGA+ 的投影机，也会有可能发道撕裂现象。最好的方案是使用的显示设备不仅仅参数相同而且型号也完全一样。网络远程访问程序如 VNC 与 LogMeIn 同样也是导致撕裂现象的原因。

关于撕裂有个非常重要的问题：没有固定的解决方法。

有时，设置不同的分辨率和刷新频率可以正常运行不会产生撕裂现象。但是，有时所有显示设置都一致时却产生撕裂现象。即便做了充分的预防措施却也无法避免撕裂现象。当撕裂现象出现时，唯一可以做的就是一步一点的分解和分析系统，找到是哪里造成的。

在 Derivative wiki 的 “Tearing” 页面有一些常见的调试方法：

通常跟着步骤操作，但没有特定的顺序：

- 核实项目没有丢帧，有时丢帧会触发撕裂。
- 核实没有任何应用程序中断或镜像显卡驱动，如 VNC 和 LogMeIn。
- 断开与电脑连接的所有显示设备，然后一个一个的重新连接直到撕裂现象发生。然后孤立这个显示设备，找出原因是由于这台显示设备的问题还是布局的问题。
- 在 NVIDIA 和 AMD 控制面板，核实所有的显示设备都设置相同的分辨率、颜色位深度，刷新频率。
- 核实显示设备的窗口没有设置旋转，这可能会导致不可知的行为。
- 检查显卡驱动版本，如果有必要则进行更新。
- 检查外部视频分配器的驱动或固件更新，如 Datapath X4 或 Matrox TripleHead2Go
- 确认只有一个 Window COMP 被渲染
- 核实 Windows Aero 被禁用。在 Windows 7，Aero 可能导致丢帧或卡顿，但不会撕裂。一旦禁用，系统可能出现撕裂，但是可以保证播放流畅。
- 在英伟达显卡中的控制面板将所有显示器进行融合，输出单屏画面。

许多情况下系统运行很正常，不会产生丢帧，但是偶尔会出现卡顿。这可能是因为显示设备与显卡刷新频率不一致导致。

如果一个项目运行的 FPS 为 30，但是显示设备的刷新率为 60Hz，有时就必须进行帧加倍。大多数情况下显卡和显示设备的这种转换是透明的，但有时就可能会出现问題。

问题发生的原因不是为每一帧进行适当的帧加倍操作，而是可能这一帧显示 1 帧时间，但下一帧显示 3 帧时间。

从项目的角度来看，没有丢失时间，没有丢帧，它不会在性能监视器或 Windows 任务管理器中报告。

如果发生类似这样的情况，使用 Window COMP 的 “FPS is Half Monitor Refresh” 功能。它会通知显卡驱动每帧应该刷新 2 次。

10.5 Edge Blending

视频投影机的使用非常灵活，可以创建无限大的画布。通过边缘重叠融合在一起使用投影机阵列创建无缝画面。这种融合重叠部分的技术被称为“边缘融合”。

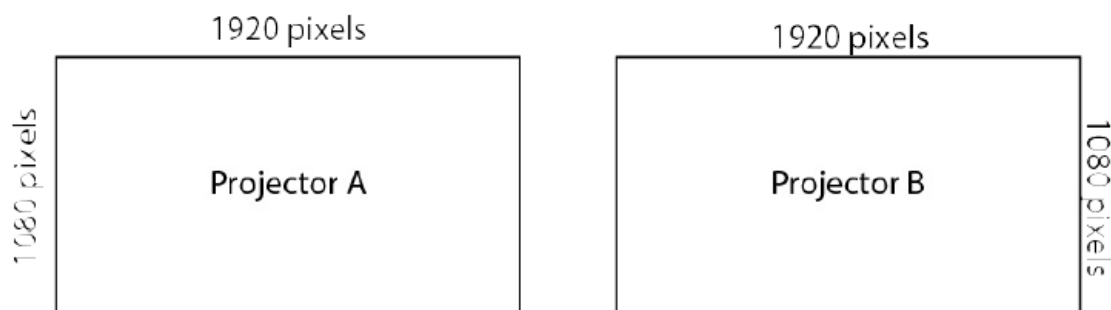
边缘融合这一概念对于那些经常工作于监视器或屏幕阵列的人来讲比较新颖。有一个经典的例子可以快速解释边缘融合背后的原理。有两台投影机，一台蓝色投影机，另一台绿色投影机。这两台投影机分别投与各自匹配的颜色，但如果你使两个颜色发生重叠，在颜色重叠的区域你得到的是绿色。这个物理原理与颜色添加混合效果是边缘融合的基础概念。

这里有一篇由 Paul Bourke 写的优秀论文，其关于边缘融合的讨论比我们这里讲的要更深入。为进一步参考与阅读，可以在下面的链接找到论文：

Edge blending using commodity projectors by Paul Bourke

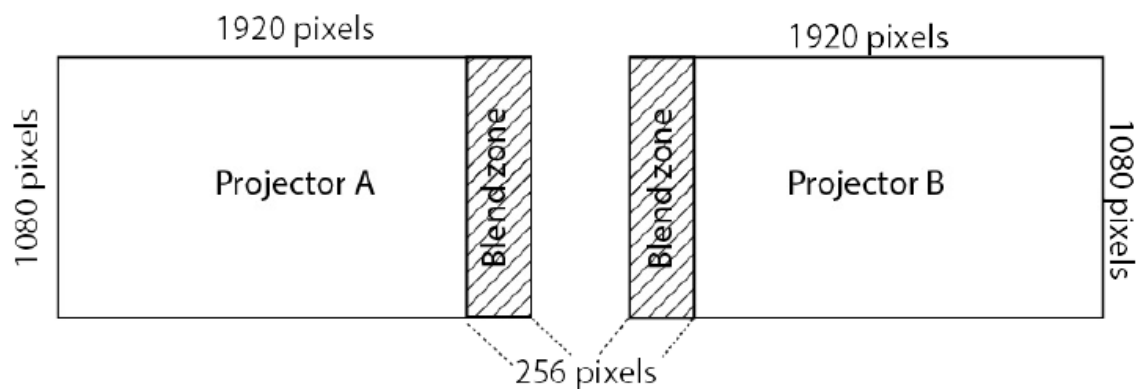
http://paulbourke.net/texture_colour/edgeblend/

学习边缘融合的基础知识最好的方法是通过案例设置。在这个案例中，目的是将两块 1920*1080 的投影机混合在一个 1*2 (1 个单位高和 2 个单位宽) 的阵列。下面是 1*2 投影机阵列的图示：



两台投影机的融合行为需要一块重叠部分。根据不同的因素，如投影机的分辨率和安装参数，所需的重叠量将有所不同。以接近单台投影机 10 %大小的 2 的指数值作为开始融合区域的试验是很好的起点。对于这个例子，1920*1080 屏幕的 10%长度是 192 像素，与其最接近的 2 的指数值是 256。

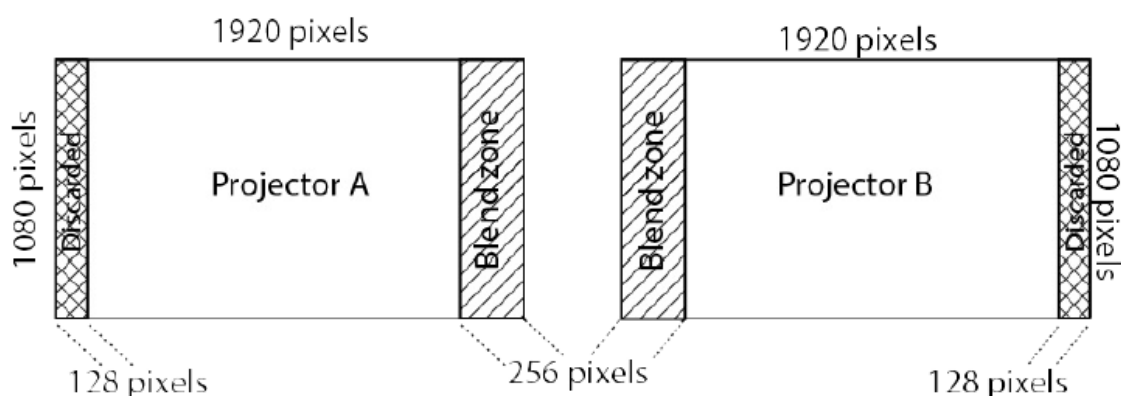
如果它的所有影响不被考虑，此重叠部分可能会导致问题。对于重叠图像，它意味着两台投影机必须在融合边缘处有相同的内容。在这个案例中，这意味着投影机 A 的右边缘与投影机 B 的左边缘必须有完全相同的内容。用数字来说明，也就是说投影机 A 的右边缘的 256 像素必须与投影机 B 的左边缘的 256 像素完全一样，如下图所示：



在完整的生产工作流程中考虑到这一事实是很重要的。我们很自然地想到一个 1*2 的阵列中两台 1920*1080 投影机需要创建的内容为 3840×1080，但这种假设可能会导致不良的后果。

投影机和显示器一样有一定的物理像素总量。两个 1920×1080 的图像挨边缘放置将有 3840 像素的宽度，但在重叠的区域像素需要增加一倍，这些用于融合的额外像素不会无中生有。融合区域将需要消耗那些用于呈现内容的像素。

在这个案例中，如果内容创建团队创建 3840*1080 的内容，256 像素将需要从某处丢弃，使得在重叠区域的 256 像素增加一倍的像素量。具体丢弃哪部分像素依偏好或工具进行设置，但是首选的方法是从每个非融合区域的边缘各丢弃一半。这种方法可以保持画布的绝对中心在融合区域，如下图所示。



当进行边缘融合时要意识到在被丢弃的像素区域应该避免放置关键信息或内容。那么为什么要在每一边删除一半的融合区域宽度（128 像素），而不是整个的融合区域宽度（256 像素）呢？答案是，要在中心创建一个 256 像素的重叠区域，每边需要偏移 128 像素到另一边。当每边偏移 128 像素时，就会产生 256 像素的重叠结果。

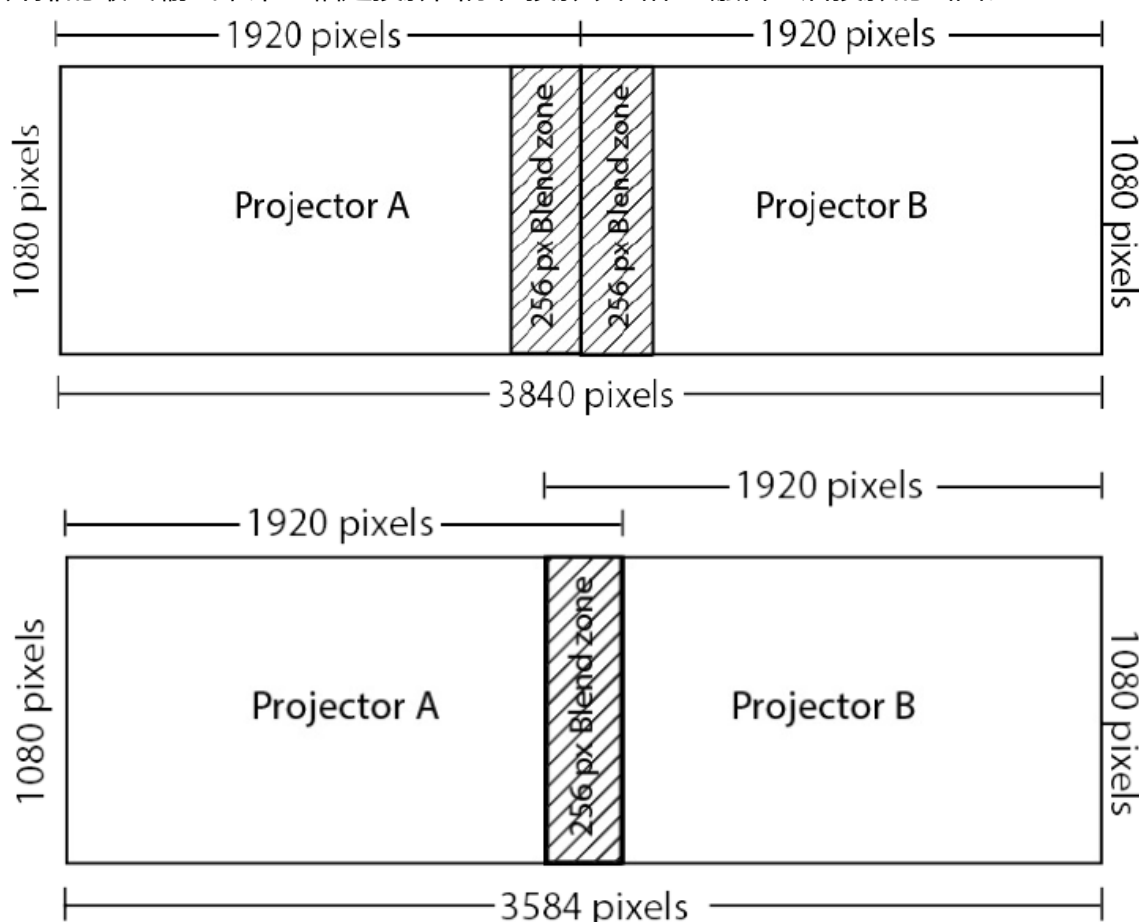
在 Touchdesigner 中做边缘融合之前，有一些方法用于创建投影机的融合内容。

首先是在完整分辨率下创建内容，在这个案例中是 3840*1080，其中边缘的一部分像素被丢弃。即使缓冲区部分要被丢弃，它也为在安装期间改变融合区域的尺寸与设置带来一定的灵活空间，而不必担心切除太多内容导致没有足够的内容填满画布。

这是项目中为没有投影机使用经验的员工在安装投影机前计算融合区域的一种“保险”手段。

若项目中有放映员或其他有经验的员工，可以让内容团队创建内容尺寸与你的可视图像尺寸一致，而不是整个的输出分辨率。这意味着在生成流程中运算更少的像素或从硬盘驱动中读取更少的像素。要计算这个案例中画面的总尺寸，用整个输出分辨率（ 3840×1080 ）减去融合区域（256 像素），会得到 3584×1080 的内容或图像分辨率。但使用这种方法存在一定的风险，如果融合区域没有计算正确，若不进行缩放，那么该内容不足以填满整个画布，

要选择哪种方式，下图中显示，第一幅是 Windows 和 TouchDesigner 中看到的每台投影机并排的最终输出，第二幅是投影图像在投影表面作过融合之后投影的画面。



在 TouchDesigner 中使用内置的边缘融合工具可以轻易实现案例中的效果。在使用工具之前，让我们来通过一个简单的案例了解边缘融合的工作流程。

打开“Simple_blend.toe”。在这个案例中，“content” Container COMP 通过拉伸 TouchDesigner 案例文件创建 3840×1080 的内容。这个容器的输出被 Crop TOP 分割为两部分，用于指定给每台投影机。这里需要注意的是前面提到的偏移量要在 Crop TOP 中进行设置。投影机 A 的 Crop TOP 从左边缘向右偏移 128 像素，从非融合边缘（在这种情况下，是左边缘）丢弃 128 像素（融合区域的一半宽度）。投影机 B 的 Crop TOP 从右边缘向左偏移 128 像素，从非融合边缘（在这种情况下，是右边缘）丢弃 128 像素。纹理各自偏移 128 像素，从而创建 256 像素的融合区域。

要创建融合区域，这些纹理乘以与融合区的尺寸 256×1080 像素一致的 alpha 渐变梯度。如果要创建一个无缝的图像还需要一些其他元素，但这个例子主要演示工作流程。

在这些元素的边缘为融合作好准备之后，可以把它们适当地合成至拥有完整输出分辨率的单一纹理中，或者在这种情况下，为每个纹理匹配与投影机分辨率一致的 Container COMP 作为背景。然后给这些 Container COMP 的设置父容器，使用父容器的 “Align Layout Horizontal Left To Right” 参数进行匹配，从而有效地创建主输出。

在这个基础案例中会丢失一些元素信息，如 Gamma 值与亮度校正，但是它能给你一个概念，让你知道程序是如何对内容进行切割和设置偏移量，之后创建重叠，以渐变梯度创建融合区域，然后创建输出。对基本的工作流程有了一定了解之后，让我们看看另一个例子，使用功能更加丰富的内置边缘融合工具。

打开 “Full_blend.toe”。这个项目文件中有两个例子，对应于之前讨论过的两种创建方法。要为你的项目中添加边缘融合组件，使用 “Alt + L” 或者在屏幕顶部的 “Dialogue” 菜单点击 “Palette Browser” 打开 Palette Browser。

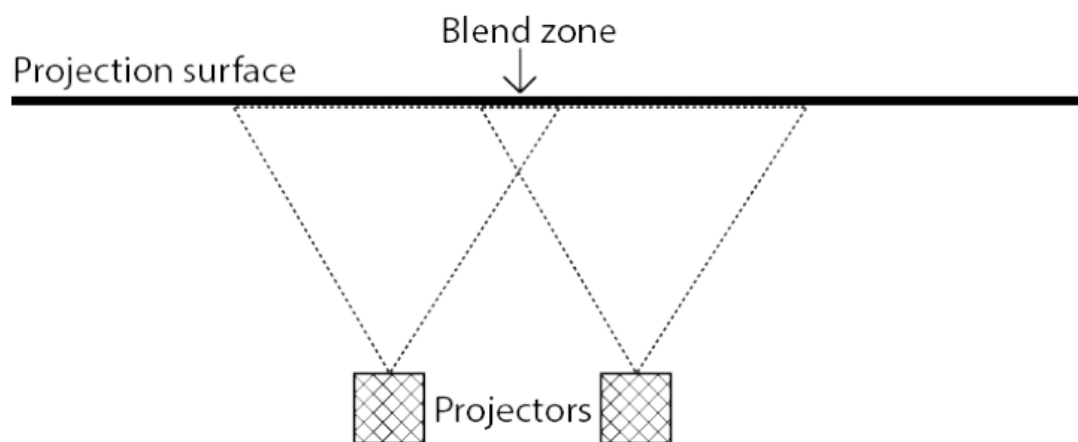
在 Palette Browser 的 “Tools” 栏目下找到 “EdgeBlend” 组件，把它拖动至你的项目中。这个组件基于之前提到由 Paul Bourke 写的同一篇论文创建。在这两个案例中 “EdgeBlend” 组件设置有 256 像素的融合宽度。“EdgeBlend” 组件在纹理中心创建重叠区域，非融合边缘丢弃额外的像素。用渐变梯度与它相乘，把处理的纹理合成至在完整投影分辨率的画布上。

第一个例子使用 3840×1080 的完整输出分辨率。这是之前提到的一个比较保险的方法，非融合边缘那些即将被丢弃的内容为融合区域在安装过程中做调整增加更大的灵活性。

第二个例子是基于 3584×1080 的图像分辨率。在这个分辨率下，不需要从纹理中丢弃像素。要记住的一个关键是，许多边缘融合工具会假设你使用的是第一个例子的路线，以完全输出分辨率提供内容，但会丢弃在外部边缘的像素，并返回与你的输入纹理分辨率相同的纹理。这就是为什么在这个例子中，图像被放入 “EdgeBlend” 组件之前，先将图像的分辨率（ 3584×1080 ）资产合成在一个完整的输出分辨率（ 3840×1080 ）的画布中心的原因。这是因为 “EdgeBlend” 组件将在中间融合区创建双倍的内容，之后再图像的一边将空白部分删除。

输出的差异可以清楚地看到，第一个例子失去调节输出边缘图像的颜色条，但是第二个例子没有。

当这些输出准备好之后，如何将正确的纹理发送给正确的投影机是一个问题。如何操控投影机与处理复杂的操作超出本章节的范围。在理想的情况下，这两台投影机的投影表面是平行的，如



如果已知融合区域的像素宽度，在“EdgeBlend”组件的“Region”参数输入它。然后尝试关闭“Enable Blend”按钮，确保你的图像正确重叠。此时，重叠画面应该匹配并被完整重叠。要时刻记住一点：边缘融合工具的主要目的是删除接缝，在内容上不会有重叠。

如果不知道融合区域的像素值，那么关闭“Enable Blend”按钮，键入一个估计数值。尝试在“Region”参数输入一个较小的值，将每个投影机的纹理向中心偏移，而一个较大的值将它们向边缘偏移。持续增加或减少“Region”参数值直到你的内容在重叠区域是相同的，然后打开“Enable Blend”按钮。

在此时，使用“Blend”滑块调整融合强度。一个较大的值将创建比较陡峭的渐变梯度，而一个较低的值将创建更加平缓的渐变梯度。

你可能需要对融合区域的内容进行 Gamma 校正。这是因为灯光不是线性变化，所以输入 0.5 的像素亮度可能不是 0.25 的两倍（像素值被标准化至 0-1 的值）。几乎所有的视频数据已经通过 Gamma 校正，但由于投影机技术的差异，实际上两个投影机的重叠部分，你可能会发现需要作出细微的调整。这些滑块是用来补偿或减少 Gamma 校正的强度。

“Luminance”滑块控制融合区域的整体亮度比图像的其余部分更亮还是更暗。使这个滑块在 0.5 以下将使融合区域更暗，而提高这个滑块在 0.5 以上将照亮融合区域。

11 优化

11.1 Introduction

创作实时项目的遇到挑战是难以预料的。除了所有的硬件与软件，还要有极为精准的实时数据。当项目运行的 FPS 为 30 时，每次的处理、显示与创建必须在 33 毫秒时间内完成。这甚至不到十分之一秒。当 FPS 越高时，这个时间越小。运行在 FPS 为 60 的项目，每帧的渲染从开始至结束只有 16ms。

意识到这是多么短的时间，因而珍惜每一毫秒的每一部分是多么重要。要清楚的知道元件的烘焙为何要花费一毫秒，慎重处理，尽可能地节省时间，知道每一毫秒都是有用的。而要做到这些需要项目分析与优化的基础技能。

TouchDesigner 使用到大量的 CPU 与 GPU 资源，能够找出是哪里需要这么多的资源是很重要的技能。当面对大型网络时，知道系统在哪里停滞，以及如何优化元件避免掉入这些陷阱，将决定项目能否成功交付。

11.2 找到瓶颈

将计算机看作一条流水线，里面的 CPU、GPU、内存与硬盘共同生产最终的产品。它们单独执行非常具体的任务，因此有时单独工作，但通常它们之间互相合作。在一条流水线上，系统运行速度只能和最慢的地方一样快。因为这种依赖性，即便其他的环节都是完全正常的，这个环节也会拖慢整个项目。这种在流水线中的薄弱环节，被称为瓶颈。

举一个瓶颈的案例，在项目中要渲染一些基础的三维几何体，为它们的表面贴上视频素材。假设这个项目包含 4 个 Box SOP，在它们的每个面贴上 1920*1080 HAP Q 格式的视频素材。电脑配置 32G 内存，8 核双芯 CPU，顶级的 NVIDIA Quadro 显卡和一块 5400-RPM 硬盘。

启动后，在这样的电脑系统上项目是运行不起来的。不管有多少内存、多少 CPU，以及多么昂贵的显卡，这个项目不能从 5400-RPM 硬盘中一次性读取多个 HAP Q 文件。因为硬盘转速不够快，不能同时读取多个 HAP Q 文件，电脑将停滞。HAP Q 文件对硬盘的要求比较高，无论电脑的其他部分多么强大，这个项目也不会运行。如硬盘不能处理像素一样，GPU 不能从硬盘读取文件。在这种情况下，硬盘就成为这个项目的瓶颈。

通常，瓶颈会出现在这三个地方：GPU、CPU 和硬盘。

GPU 本身就是一个流水线，像素着色阶段很可能成为瓶颈。每当操作像素，使用几乎所有 TOP，系统需求更多的 GPU 着色器。越高的分辨率，越高的 GPU 需求。TOP 的分辨率与 GPU 运算量呈 1:1 的正比关系。如果 TOP 分辨率降低一半，它的 GPU 运算量也会按比例降低。是否有像素着色器瓶颈的一个快速检查方法是降低所有生成 TOP（如 Render TOP 与 Constant TOP）的分辨率。如果在速度与性能上有明显的增加，显然存在像素着色器瓶颈。

当显卡过载，看似随意的 TOP 将开始耗费比普通渲染更长的时间。在 Performance Monitor 中，TouchDesigner UI 元素的渲染时间可明显观察到。如果所有的 UI 元素的渲染时间突然超过 1 毫秒，就需要优化网络减轻 GPU 在这一块的负担。

从经验来看，CPU 是瓶颈出现的第二个地方。所有的元件都需要 CPU 运行，因此 CPU 很快会过载。可以在 Performance Monitor 可以看到元件消耗大量的时间渲染，因而 CPU 的瓶颈往往比较容易跟踪。用 Hog CHOP 可以衡量有多少 CPU 的预制时间（headroom）。这个 CHOP 的作用与其名称一样，占用 CPU 处理。其“Delay”参数设置每一帧渲染总时间。在创建 Hog CHOP 后，项目开始丢帧，就说明 CPU 是我们的瓶颈。

所有的影片文件使用 CPU 从它们的压缩状态解码数据。某些编码器对 CPU 的使用比其他多，读取多个影片文件时使用的 CPU 资源比想像中的多。

过载的 CPU 反应类似于过载的 GPU，在 Performance Monitor 会出现不一致的结果。元件呈现不同的烘焙时间。这是因为它们开始操作后，在它们完成之前，CPU 又被叫去执行其他过程。元件花费时间等待 CPU 回来执行它的过程因而增加它的烘焙时间。与 GPU 过载不同，CPU 过载不仅仅影响 TOP。

硬盘驱动器是瓶颈出现的第三个地方。特定操作会对硬盘的要求比较高。当为部署项目配备系统时容易忽视高质量的固态硬盘（SSD）。取决于使用的编码器，在硬盘上读写影片的操作会快速耗尽硬盘性能。这样的瓶颈提示会出现在 Performance Monitor 中 Movie In TOP 行下，如：

```
Waiting for frame for 90 - E:/test.mov from harddrive
```

数字是影片的帧，路径是影片的文件。

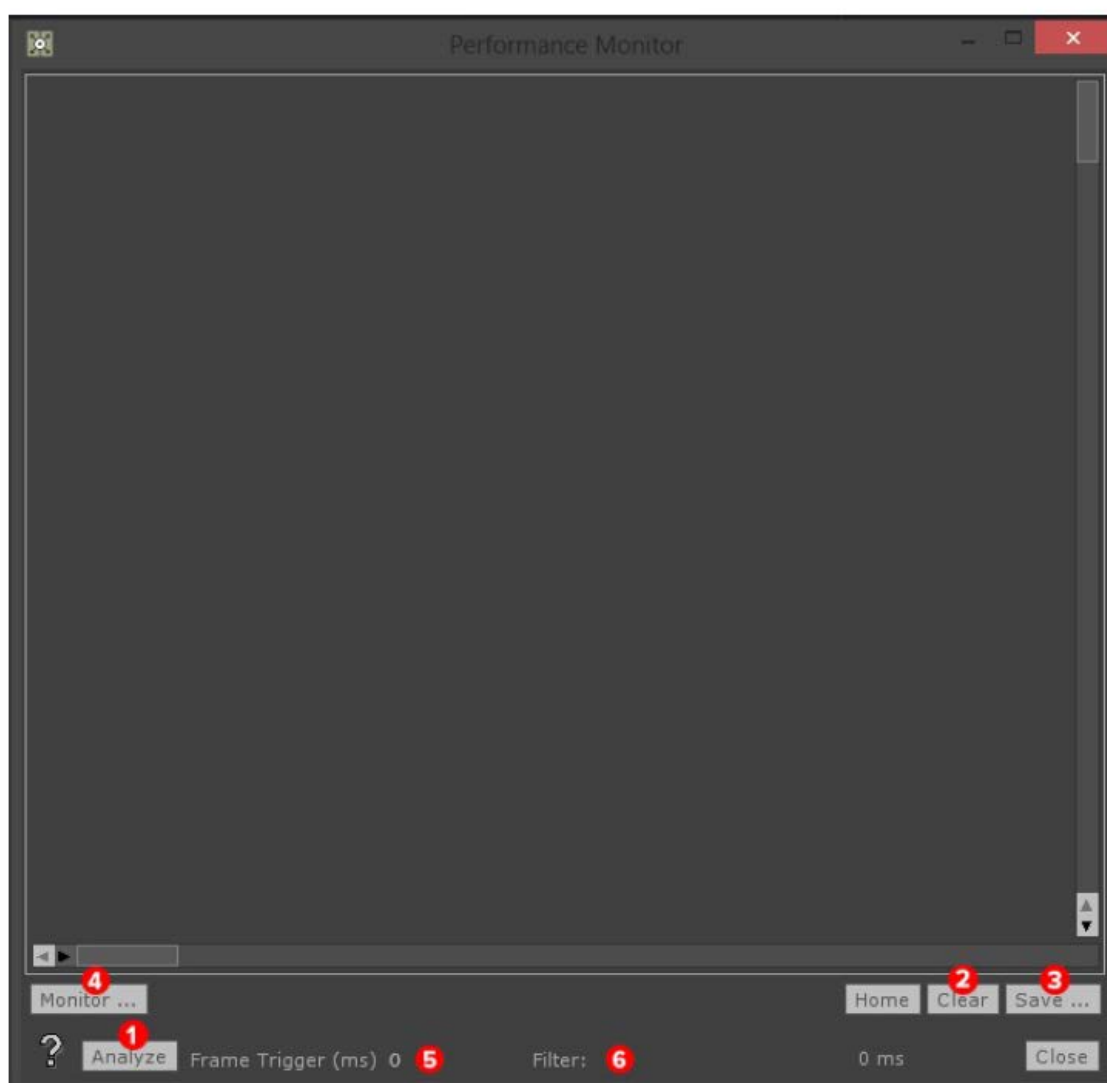
11.3 Using Performance Monitor

Performance Monitor 是分析帧烘焙时间的工具。当要进行优化或调试项目性能时很有用。

有三种方法访问 Performance Monitor。

1. 键盘上按 “F2”
2. 快捷键 “Alt + Y”
3. 在屏幕顶部菜单 “Dialogs” 下拉列表中选择 “Performance Monitor”

它有几个按钮，功能一目了然。

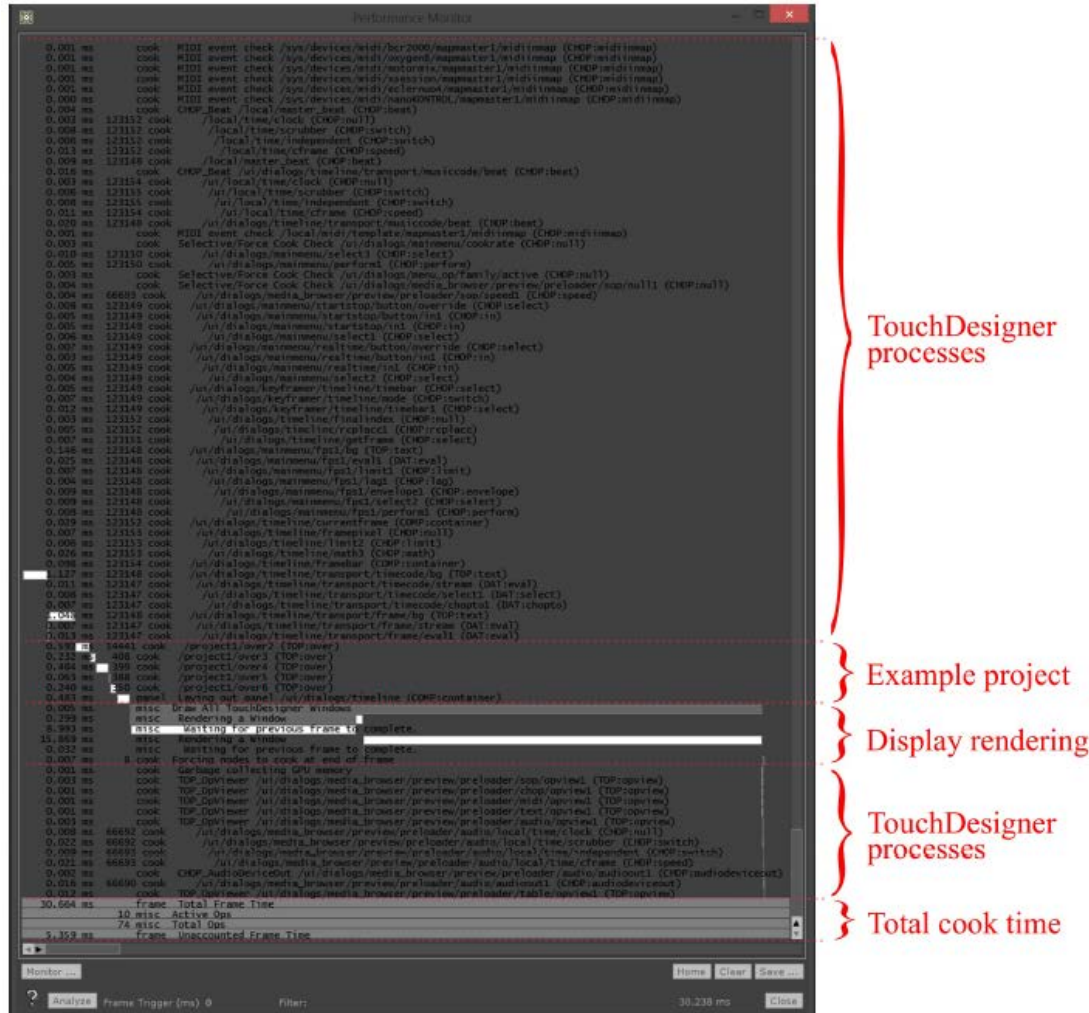


1. 执行分析当前帧
2. 清除当前结果
3. 保存当前结果至文件文件
4. 改变监视选项
5. 毫秒阈值，如果烘焙时间到达，触发 Performance Monitor 进行分析
6. 更加精确地筛选结果，如只查看 CHOP 或 TOP 等。

Introduction To TouchDesigner

需要注意的是，烘焙时间基于 CPU，并不意味着 GPU 瓶颈会在 Performance Monitor 中被忽视，但是在 Monitor 中我们要意识到这是 CPU 的数据。

这是一个来自 Performance Montior 分析结果的例子：



上面的分析来自一个简单的项目，它包含几个 Movie In TOP 与几个 Over TOP。这些元件负责 TouchDesigner 的功能和用户界面元素。

在上图标记的部分查看渲染时间与路径，TouchDesigner Processes 中的元件负责 TouchDesigner 的主要功能（用户界面、会话窗口等等），而属于工程案例（Example Project）的元件更加重要。每个元件烘焙时间用白条代表。白条的尺寸代表它占用总烘焙时间的比例。当项目增长变得更加复杂时，“Example project”标记块会相应增长并包含项目中所有的元件。通过它来分析项目，找到占用太多系统资源的问题元件。

在这个案例中，可以跟踪 Over TOP 的路径与烘焙时间。

它们都在 “project1” 容器内，其渲染时间范围在 0.06 毫秒到 0.6 毫秒之间。

强烈提醒：注意垂直同步的影响，以及渲染时间在 Performance Monitor 是如何被计算的。显卡会尝试锁定项目的 FPS 与显示刷新率。在 30FPS，即使是一个空的项目，每帧的烘焙时间也是 33 毫秒。在 60FPS，会有同样的效果，Performance Monitor 将会显示烘焙时间为 16 毫秒。这并不是说每帧的实际渲染时间刚好是 33 毫秒或 16 毫秒，而是垂直同步尝试同步 TouchDesigner 的 FPS 与显示刷新率。

11.4 Operator Cooking

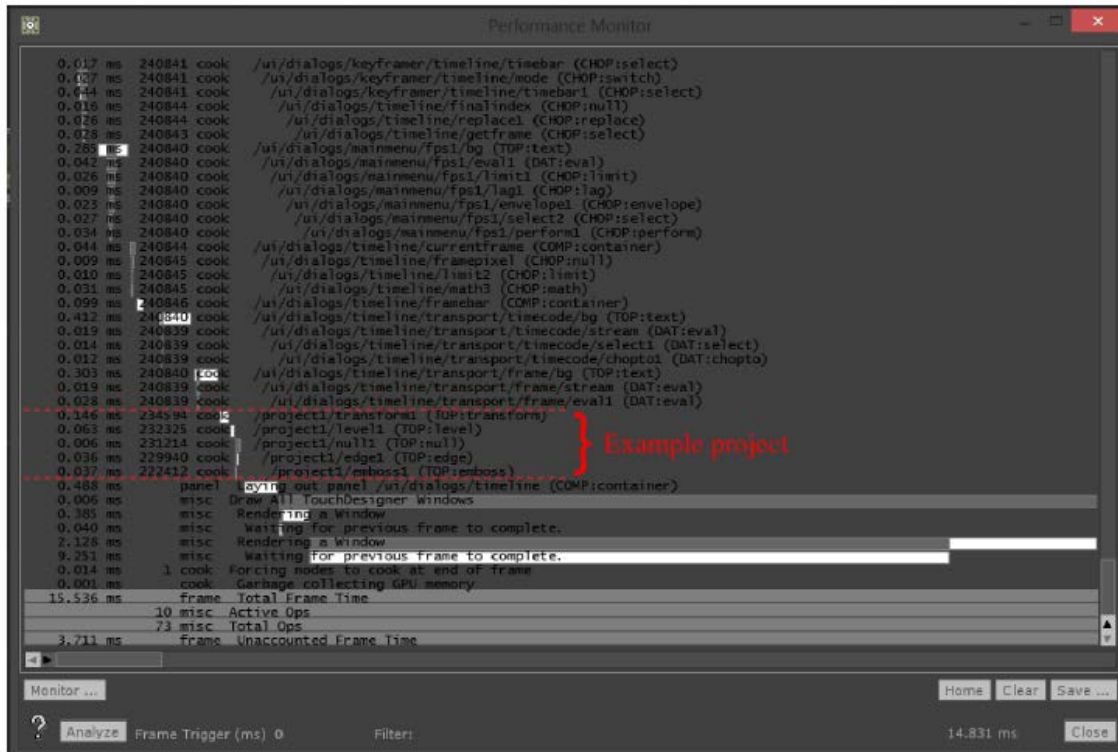
通过降低元件每帧的烘焙总时间以得到更好的性能。初学者在创建网络时可能从来不考虑到这一点。诚然，每个人都有赶工的时候，在创建网络时无法考虑太多。然而，如果一个项目要求不能丢帧，那么不考虑烘焙是非常冒险的。

主要的目标是在信息流程中尽可能早的处理静态操作以防止在每一帧都被渲染。

打开 “Cooking_1.toe”。在这个案例，旋转一个简单的图形，然后再给它其他操作使得看起来更有趣。有一个功能可以帮助优化网络：连接元件的带动画的线。动画的线意味着线的两端每帧都会被烘焙。从网络的最左边开始，连接 Movie In TOP 与 Transform TOP 的线是没有动画的。这是因为图像被载入并保持静态。元件只在它们需要被执行或改变时才进行烘焙。一个静止的图片是静态的，不做改变，也就不需要每帧都进行烘焙。

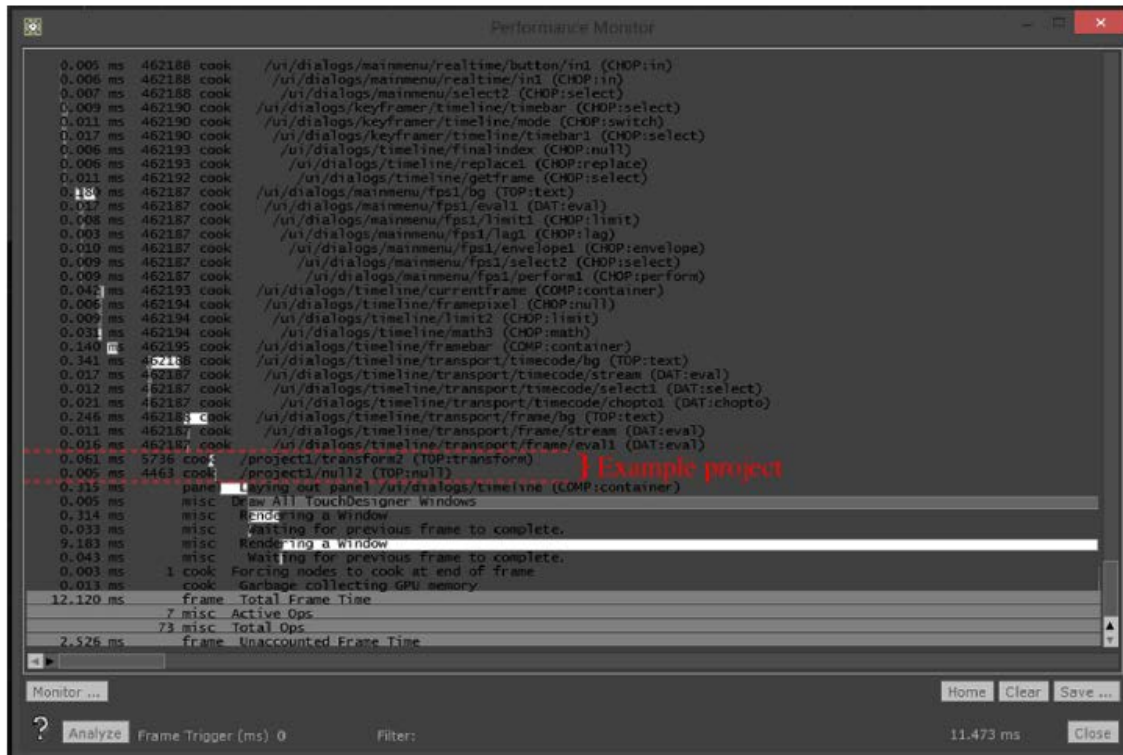
与之相反，网络中的其他线都是带动画的，意味着 Movie In TOP 之后的所有元件在每一帧都需要烘焙。对于这个项目，这不是一个可怕的问题，因为这里没有发生非常复杂的事情。从一开始就创建高效的网络可以节省优化网络提升性能的时间。让我们在 Performance Monitor 中查看这个项目。

Introduction To TouchDesigner



忽略为实现 TouchDesigner 功能必要的所有元件，有一小块展示的是这个案例。对图像执行的操作，总共花费了大约 0.25 毫秒。如前所述，静态操作只烘焙一次，要注意的是许多操作在 Transform TOP 之后都是静态的。让我们重新排列这些，再查看性能上的提升。

打开 "Cooking_2.toe" 。这个项目与之前的项目完全相同，除了元件被重新排列，在查看信息流程之前，让我们先来查看 Performance Monitor。



乍一看，似乎这个项目缩小了。有几个元件已经消失。因为这些元件并不是每帧都要烘焙。在上一个案例中，Transform TOP 在每帧都执行变换，使得它之后的所有 TOP 每帧都需要重新计算。而在这个案例中，所有的静态操作都放在信息流程的开始，留下 Transform TOP 执行它的变换，其他元件都不需要重新计算或烘焙。

更进一步查看 Performance Monitor 结果，只有 Transform TOP 元件被烘焙，而且耗费是 0.061 毫秒。

比较之前的案例，一系列的元件烘焙耗费 0.25 毫秒。简单的改变却有令人难以置信的改善。

值得注意的，两组元件的输出结果可能会不完全相同，但是如此难以区分，以至于许多客户 and 艺术家都不会介意，尤其在知道有这么大的性能提升后会允许这样做。

11.5 分辨率

在谈到平面纹理时，分辨率是非常重要的。因为每一个像素点的消耗都会带来资源的浪费。

用一个案例以一个非常很简单的方式说明这一事实。打开 “Resolution_1.toe”。有这样的一个简单的设置：将一只蝴蝶连在一个大画布上，然后使用一些 LFO（低频振荡器），在合成至森林背景之前调整它的透明度与模糊。在任意一个 TOP 上点击中会显示这个案例需要超过 100M 的 GPU RAM。在一个有 4G RAM 的 GPU 系统中这不是一个很大的空间，但是它会速度地增加。尝试实时合成 40 只蝴蝶，4G 空间很容易耗光。

现在对比 “Resolution_2.toe”。它得到完全相同的结果，但是只消耗了 60M 的 GPU RAM。这是一个显著的差异。使用第二种方法，在案例中合成 40 只蝴蝶，只占用大约 2.4G 的 GPU 内存。由此可见产生的空间差异仅仅是因为我们做出了一些小的改变。

蝴蝶的源资产只有 512*512 的分辨率，在第一个案例中，立刻合成到一块 1920*1080 的画布上，然后调整。蝴蝶被调整后，Touchdesigner 需要在每一帧重画 1920*1080 的画布。

“空”像素即使没有颜色与 Alpha 数据也同样被重画。在第二个案例，执行完全相同的操作，但只在更低分辨率的源资产上操作。把调整后的资产合成在 1920*1080 的画布。这样 GPU 只需要处理一小部分像素，节省在大画布上的重画，从而节省相当多的 GPU 内存与处理能力。

11.6 GPU 内存碎片

使用 GPU 的元件通常为它们的任务分配资源，并保持占用，直到任务完成或改变。

当运行在有多个不同内容的来源项目运行时，GPU 碎片是一个主要的关注点。举例，一个 1280*720 的 Constant TOP 元件连接到 10 个其他元件上。一旦连接并烘焙，各元件将预留出需要处理特定数量像素的 GPU 内存。一旦内存被分配，元件能够在它们的分配空间内比较高效的运行。

如果源 Constant TOP 的分辨率发生改变，这将触发一系列的连锁反应，其他 10 个元件全部需要为它们的任务重新分配 GPU 资源。如果元件的资源可以被高效分配，许多一样的内存块将被重用。相反地，如果它们不能重用内存块，而必须迁移至内存尾部。如果这样的情况接二连三的发生，GPU 内存将变得破碎，GPU 将试图整理它的内存，而项目将在此不佳的状态下运行。

下面两幅图尝试以最简单的方式描述内存碎片。

要研究的两个案例，具有相似的参数。3GB 的 GPU RAM，载入 3 张 1G 的静态纹理，并且在 RAM 中一直保持占用。

Case 1: 3GB of GPU RAM



在案例 1 中，3G RAM 的 GPU 可以完全适合三张 1G 的静态纹理。它们是静态纹理因为一旦被分配空间，就不再改变。这相当于从项目一开始就在 Movie In TOP 中载入一张大尺寸图像，然后一直就保持占用。

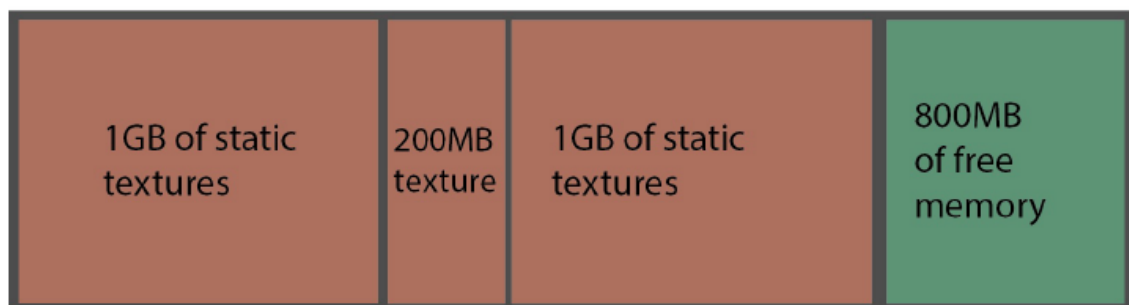
这是一个理想中的情况，因为有许多进程要使用 GPU 内存资源，意味着它是不断变化的，而且在 3G RAM 的显卡上永远不会有 3G 待分配空间。

案例 2 描述的情况将产生内存碎片。已经存在三张 1G 的纹理，再加入一张 200M 的纹理。在这个案例中，依如下顺序执行载入与卸载操作。

1. 载入 1G 的纹理
2. 载入 200M 的纹理
3. 载入 1G 的纹理
4. 卸载 200M 的纹理
5. 载入 1G 的纹理

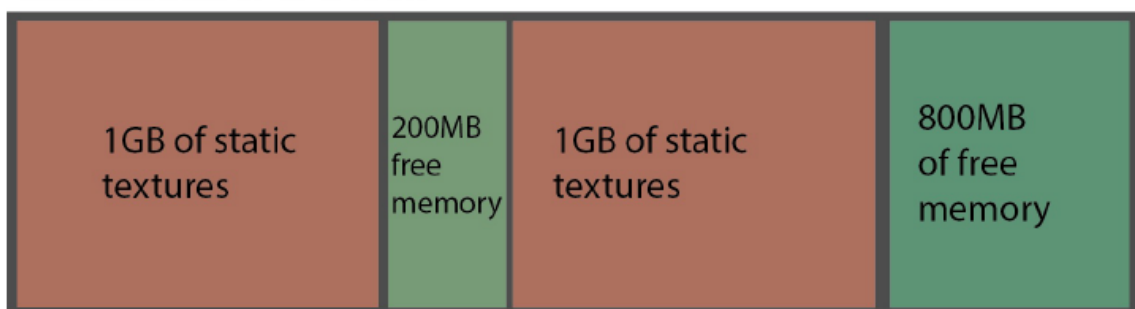
模拟这样的情况：载入与显示一张纹理，然后使用其他纹理将其替换。

Case 2.1: 3GB of GPU RAM



在图 Case 2.1 中，从第一步至第三步，还有 800M 的可用空间。乍一看，这似乎是完美的，因为 200M 的纹理被卸载，有足够载入最后纹理的 1G 空间。不幸的是，这不是显卡的工作方式。

Case 2.2: 3GB of GPU RAM



如上所述，在图“Case 2.2”中，第四步完成后，200M 纹理被卸载。这是 GPU 内存碎片的典型例子。总共有 1G 的 GPU 可用空间，但是没有一块完整的 1G 的空间以分配给 1G 的纹理。已经载入的 1G 纹理，它们处理静止状态，不能在 GPU 内存没有完全卸载和重载的过程中发生迁移。因为内存不能移动，200M 的可用空间会一直被困在静态纹理之间，这 200M 的可用空间可以被 200M 或更小的纹理使用，但不能载入第三张 1G 的纹理。

防止内存碎片严重的最好方法是尝试与限制项目中使用不同分辨率的数量。如果换成相同分辨率的资产，通常可以使用它在内存中的原来位置。

11.7 Windows System Processes

Windows 是一个复杂的系统，有许多进程和与系统相关的程序在后台运行。许多进程和程序会对计算机性能和 TouchDesigner 产生负面影响。

Windows 对于那些白天使用电脑的人有巨大的市场。正因为如此，如果一台电脑在夜间供电，有许多应用程序和 Windows 系统操作将按计划运行。对于一天 24 小时运行的演出装置，这可能会有问题，许多不同的应用程序与系统相关任务应当被关闭或重新安排。

像杀毒软件和间谍扫描软件，可以在日常的电脑中使用，通常应当避免在 TouchDesigner 的电脑中使用。它们可能会导致一些问题，第一个是弹出窗口。许多这样的软件在一定的时间间隔会弹出提醒和通知，如果比较不幸，它们会重叠在显示的内容上并覆盖演出装置的输出。这些程序对硬盘同样会产生负面的影响，因为它们经常扫描系统中的病毒与恶意软件，占用硬盘读写周期。这些都是会占用 CPU 资源的，所以需要注意那些一直在运行的程序。